

Nand to Tetris, Part II

These slides support the Introductory Section

“II: Software” of the book

The Elements of Computing Systems

By Noam Nisan and Shimon Schocken

MIT Press, 2021

Hello, World Below

High-level program (Jack)

```
// First example in Programming 101
class Main {
    function void main() {
        do Output.println("Hello World!");
        return;
    }
}
```



Issues

- How does the program execute?
- How does it handle class, function, ... etc.?
- How are classes and methods realized?
- How is function-call-and-return handled?
- How is memory allocated to objects and arrays?
- How does the program write on the screen?

Add your questions to the list...

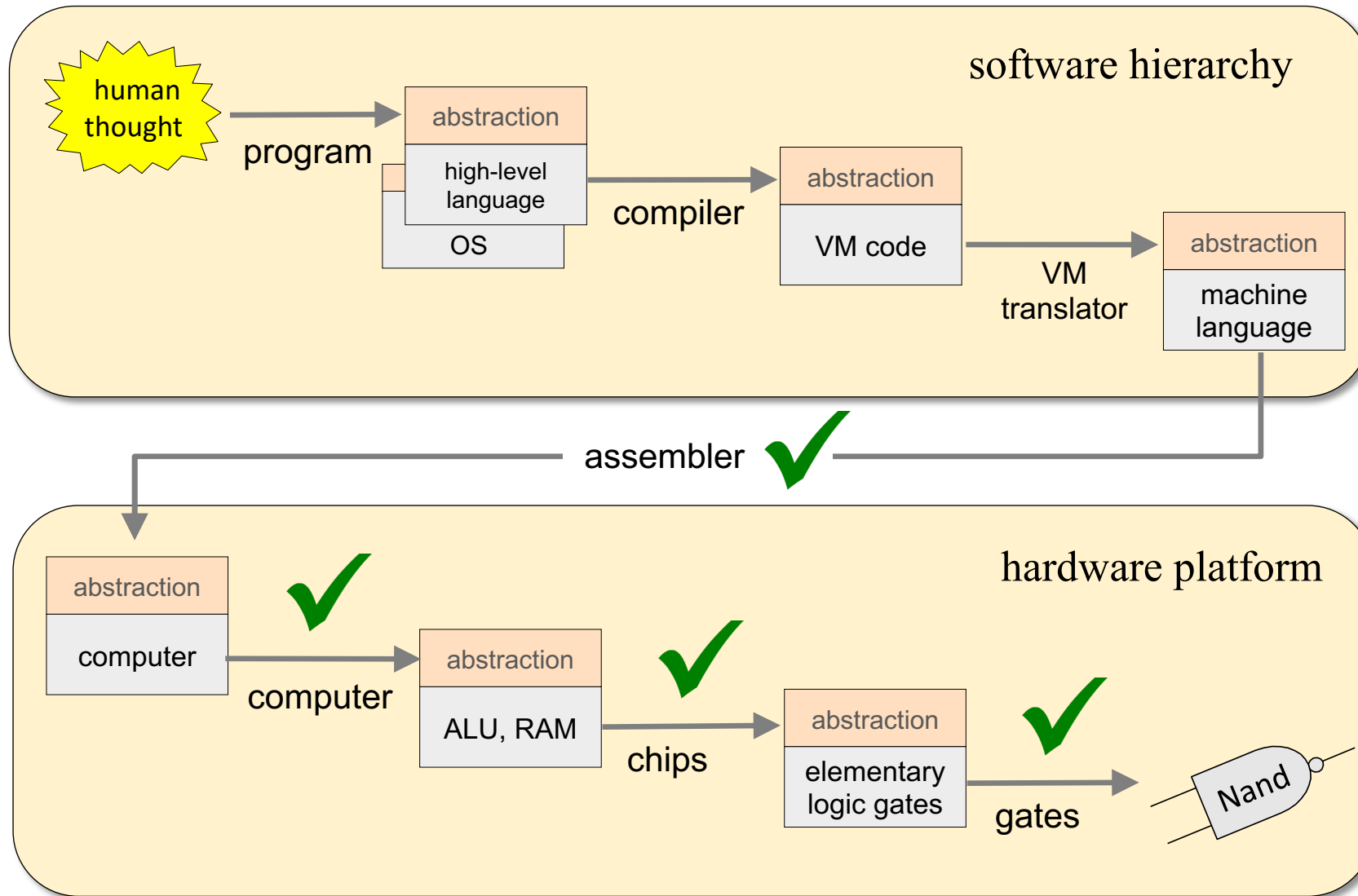
Q: How can high-level programmers ignore all these issues?

A: They treat the high-level program as an ***abstraction***

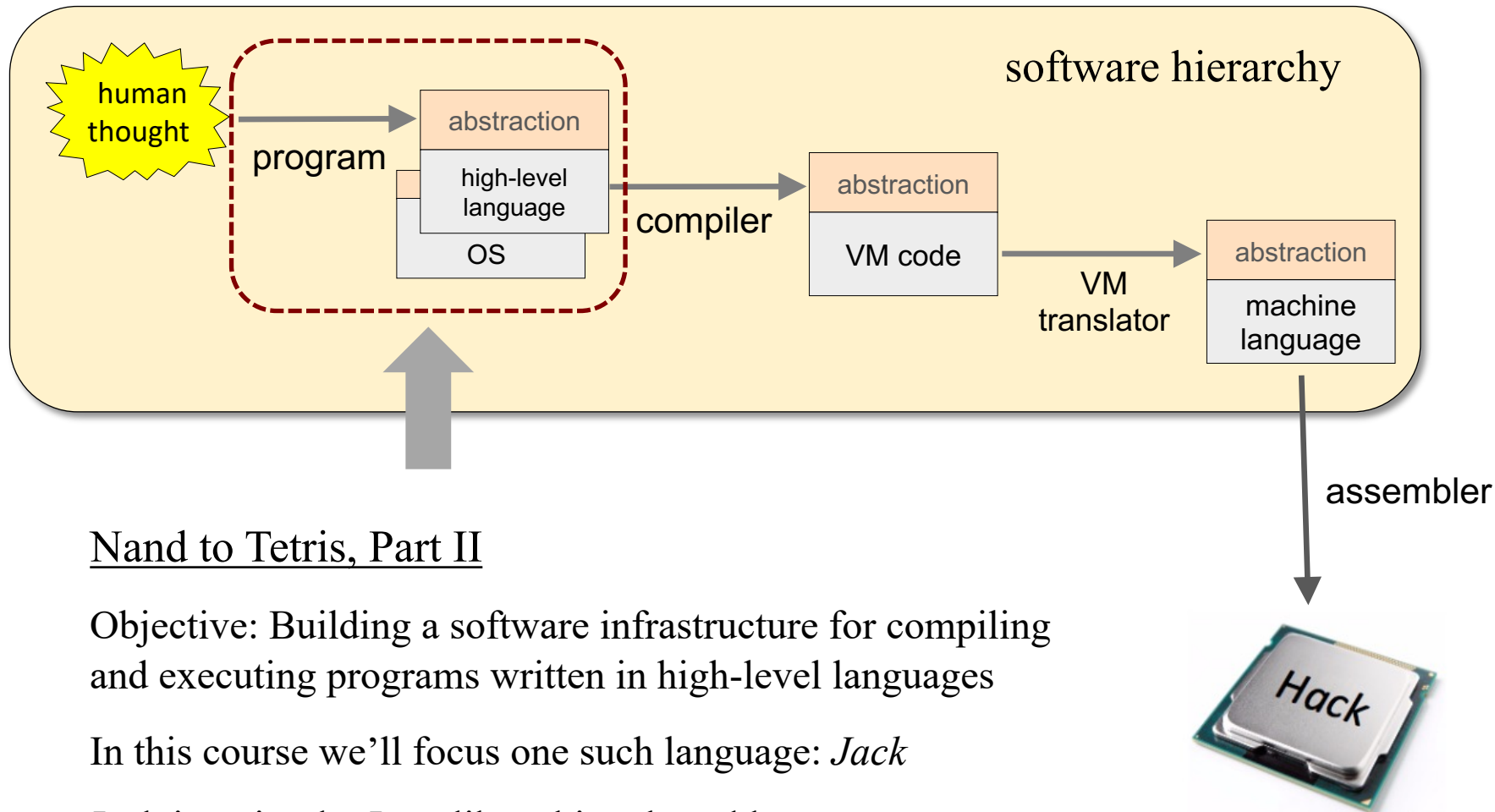
What makes the abstraction work?

- Assembler
 - Virtual machine
 - Compiler
 - Operating system
- } Projects 6-12

Nand to Tetris Roadmap: Part I



Nand to Tetris Roadmap: Part II



Nand to Tetris, Part II

Objective: Building a software infrastructure for compiling and executing programs written in high-level languages

In this course we'll focus one such language: *Jack*

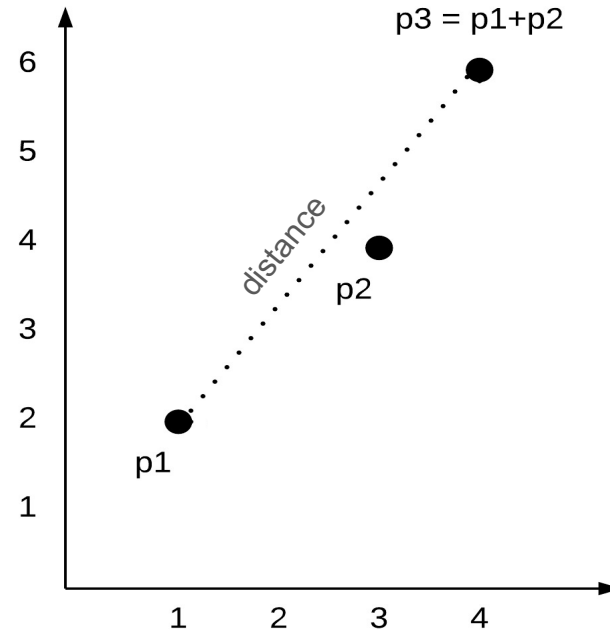
Jack is a simple, Java-like, object-based language.

High-level language

Example: write a Jack program

That uses a typical `Point` class for:

- Constructing 2D points, like `p1` and `p2`
- Computing and printing `p3 = p1 + p2`
- Computing and printing `distance(p1, p3)`



Point class (skeletal)

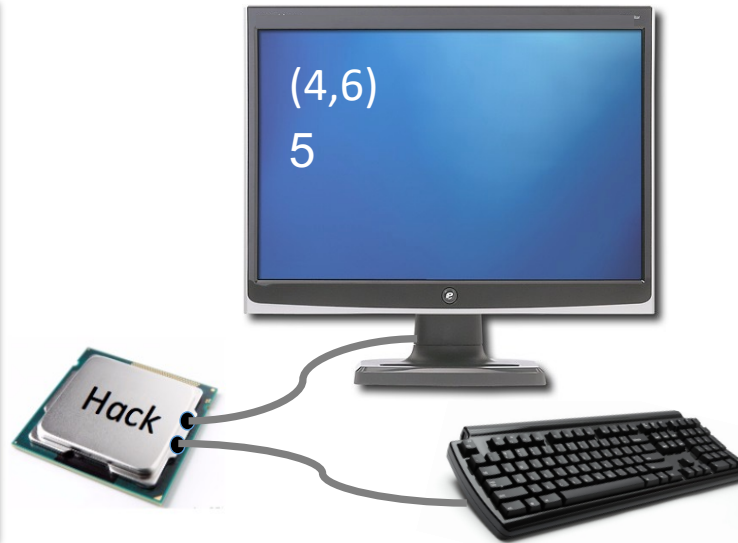
Written in the
Jack language

```
/** Represents a 2D point */
class Point {
    /** Constructs a new point with the given coordinates */
    constructor Point new(int ax, int ay)

    /** Returns the point which is this point plus the other point */
    method Point plus(Point other)

    /** Cartesian distance between this and the other point */
    method int distance(Point other)

    /** Prints this point as "(x,y)" */
    method void print()
    ...
}
```



High-level language

```
/** Constructs and manipulates Point objects */
class Main {
  function void main() {
    var Point p1, p2, p3;
    let p1 = Point.new(1,2);
    let p2 = Point.new(3,4);
    let p3 = p1.plus(p2);
    do p3.print();
    do Output.println();
    do Output.printInt(p1.distance(p3));
    return;
  }
}
```

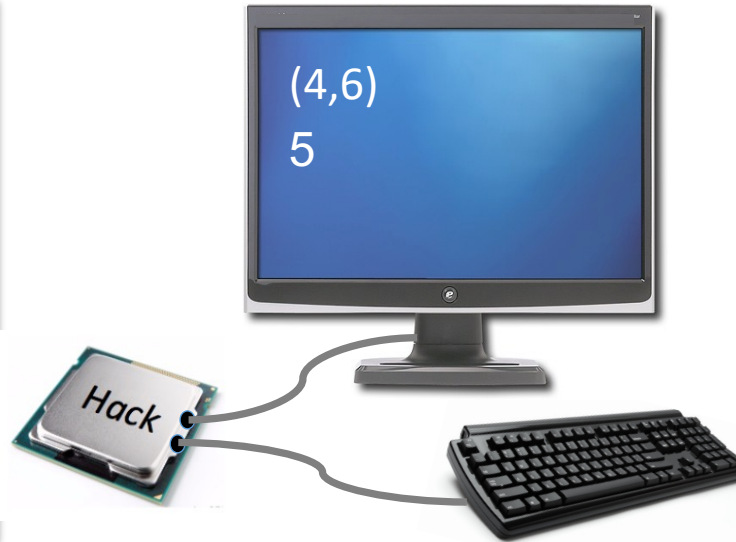
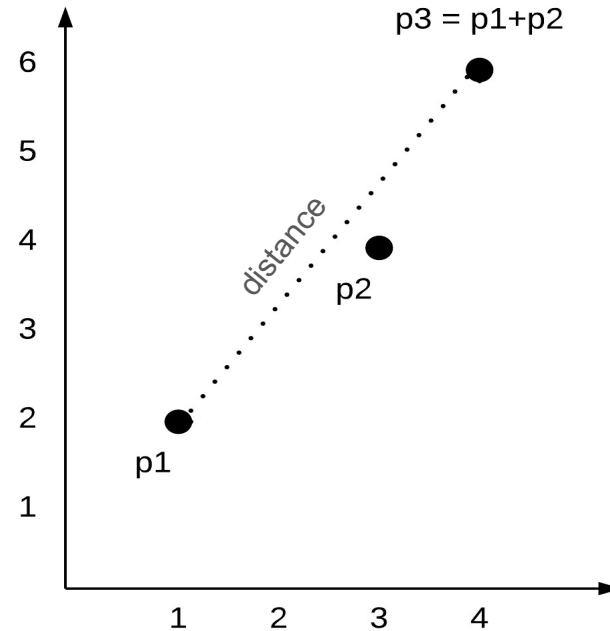
Written in the
Jack language

```
/** Represents a 2D point */
class Point {
  /** Constructs a new point with the given coordinates */
  constructor Point new(int ax, int ay)

  /** Returns the point which is this point plus the other point */
  method Point plus(Point other)

  /** Cartesian distance between this and the other point */
  method int distance(Point other)

  /** Prints this point as "(x,y)" */
  method void print()
  ...
}
```



High-level language

```
/** Represents a 2D point.  
    File name: Point.jack. */  
class Point {  
    // The coordinates of this point  
    field int x, y  
  
    // The number of Point objects constructed so far:  
    static int pointCount;  
  
    /** Constructs a two-dimensional point and  
        initializes it with the given coordinates. */  
    constructor Point new(int ax, int ay) {  
        let x = ax;  
        let y = ay;  
        let pointCount = pointCount + 1;  
        return this;  
    }  
  
    /** Returns the x coordinate of this point. */  
    method int getX() { return x; }  
  
    /** Returns the y coordinate of this point. */  
    method int getY() { return y; }  
  
    /** Returns the number of points constructed so far. */  
    function int getPointCount() {  
        return pointCount;  
    }  
  
    // Class declaration continues on top right.
```

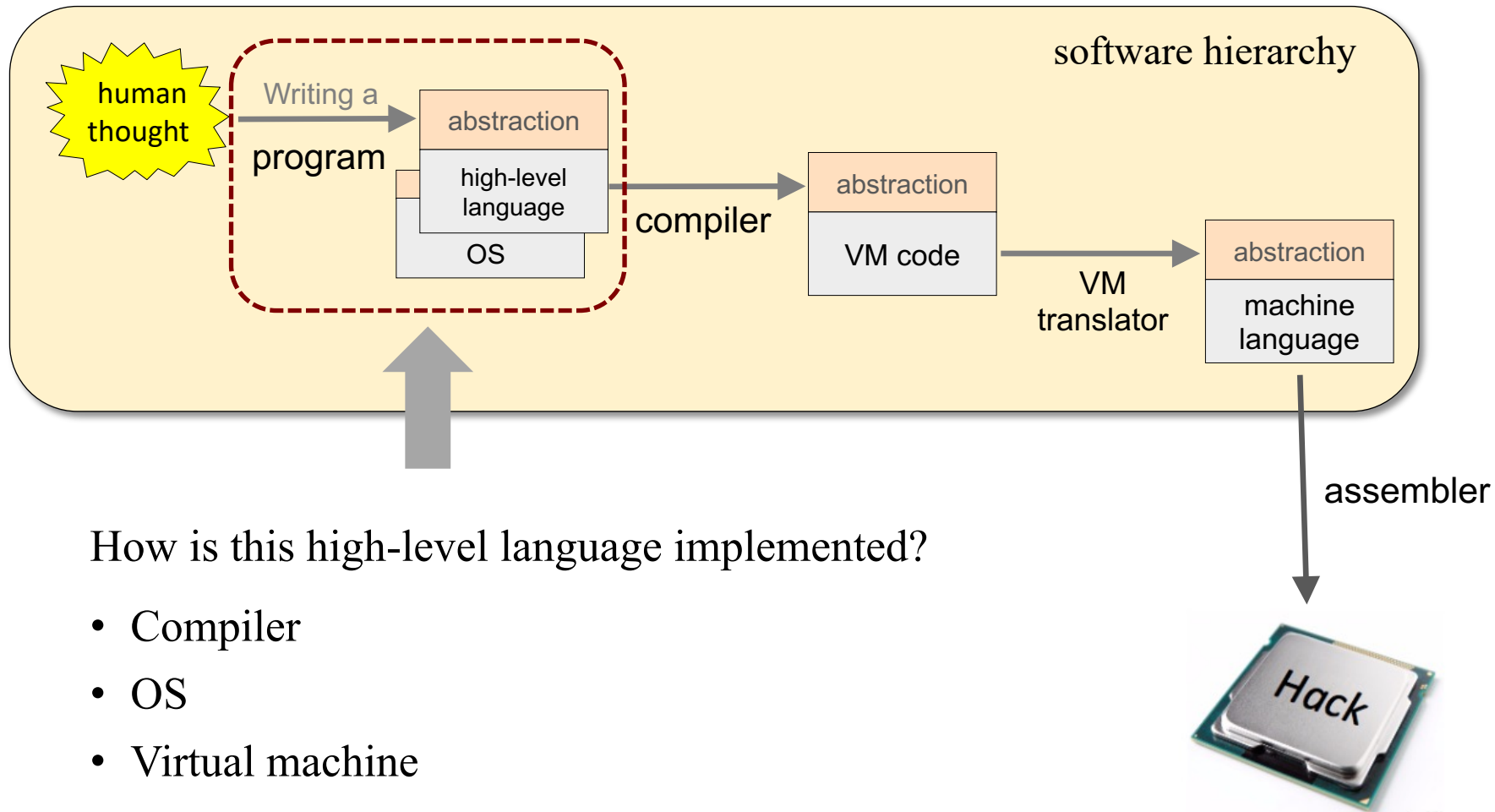
Jack is a typical object-based,
high-level language
Details, later.

High-level language

```
/** Represents a 2D point.  
    File name: Point.jack. */  
class Point {  
    // The coordinates of this point  
    field int x, y  
  
    // The number of Point objects constructed so far:  
    static int pointCount;  
  
    /** Constructs a two-dimensional point and  
        initializes it with the given coordinates. */  
    constructor Point new(int ax, int ay) {  
        let x = ax;  
        let y = ay;  
        let pointCount = pointCount + 1;  
        return this;  
    }  
  
    /** Returns the x coordinate of this point. */  
    method int getX() { return x; }  
  
    /** Returns the y coordinate of this point. */  
    method int getY() { return y; }  
  
    /** Returns the number of points constructed so far. */  
    function int getPointCount() {  
        return pointCount;  
    }  
  
    // Class declaration continues on top right.
```

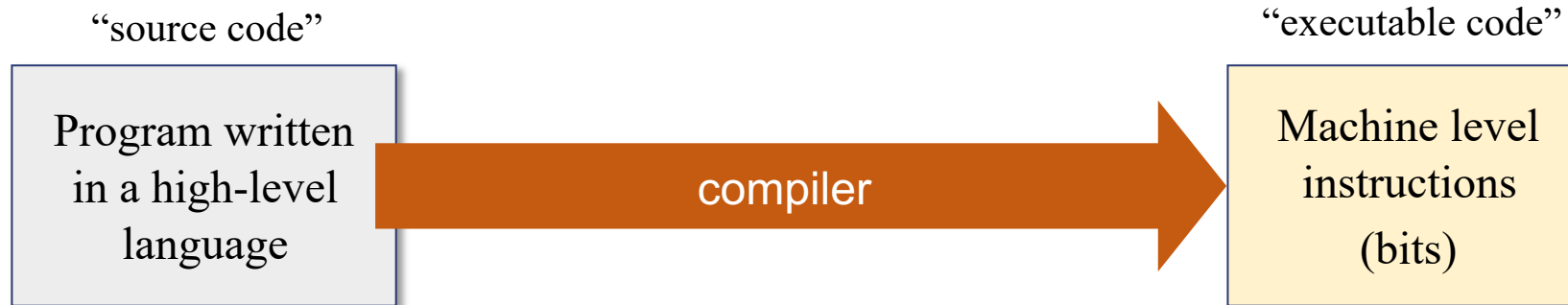
```
/** Returns a point which is this  
    point plus the other point. */  
method Point plus(Point other) {  
    return Point.new(x + other.getX(),  
                    y + other.getY());  
}  
  
/** Returns the Euclidean distance between  
    this and the other point. */  
method int distance(Point other) {  
    var int dx, dy;  
    let dx = x - other.getX();  
    let dy = y - other.getY();  
    return Math.sqrt((dx*dx) + (dy*dy));  
}  
  
/** Prints this point as "(x,y)" */  
method void print() {  
    do Output.printString("(");  
    do Output.printInt(x);  
    do Output.printString(",");  
    do Output.printInt(y);  
    do Output.printString(")");  
    return;  
}  
  
} // End of Point class declaration.
```

Nand to Tetris Roadmap: Part II

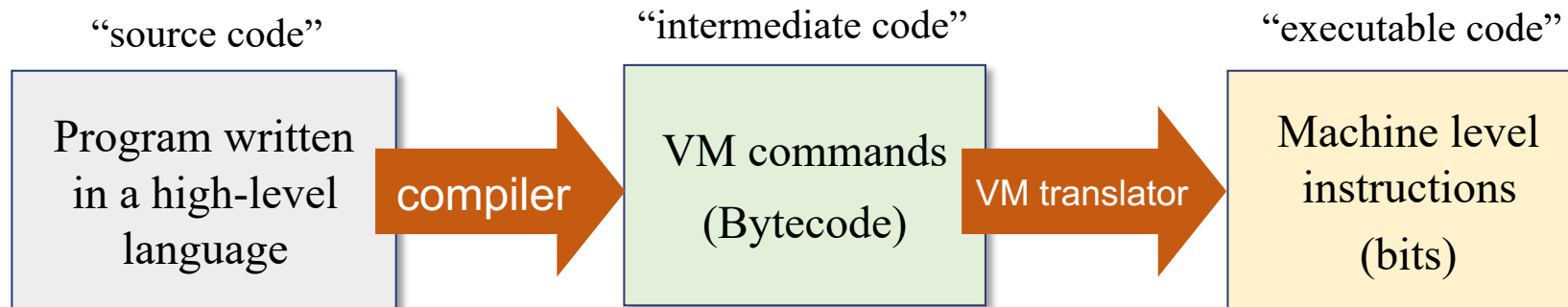


Compilation

One tier



Two tier



Compilation

One tier



Pros

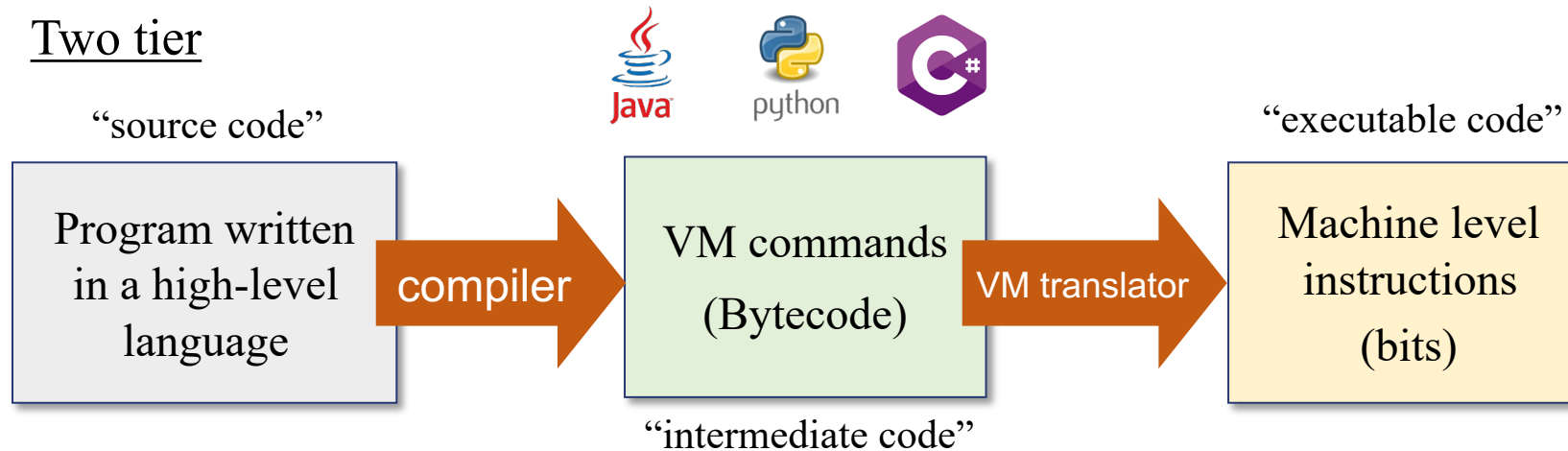
1-tier compilers generate efficient, target-specific code

Cons

Requires multiple compilers / compilations, one for each target hardware platform.

Compilation

Two tier



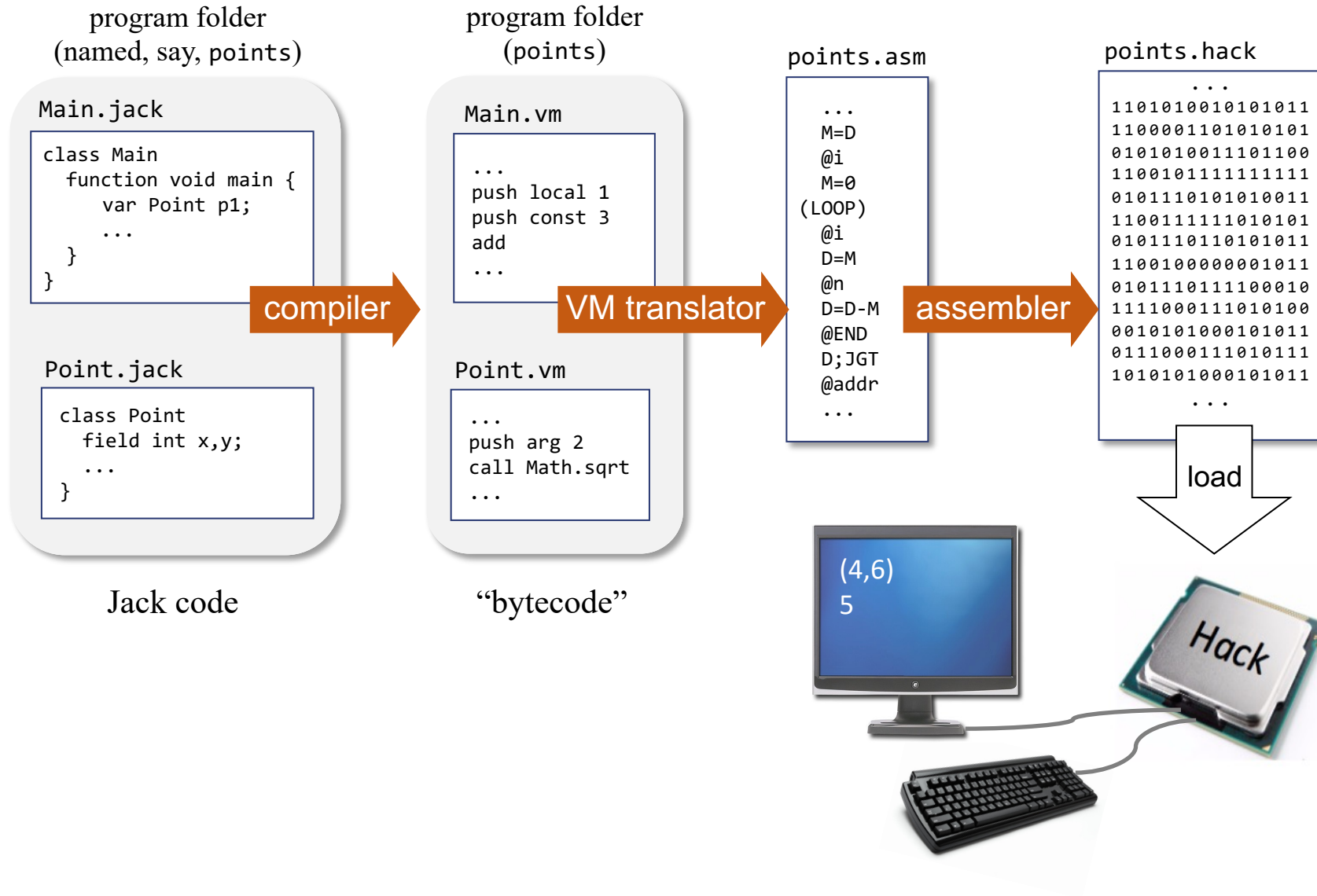
Pros

Compile once, run everywhere

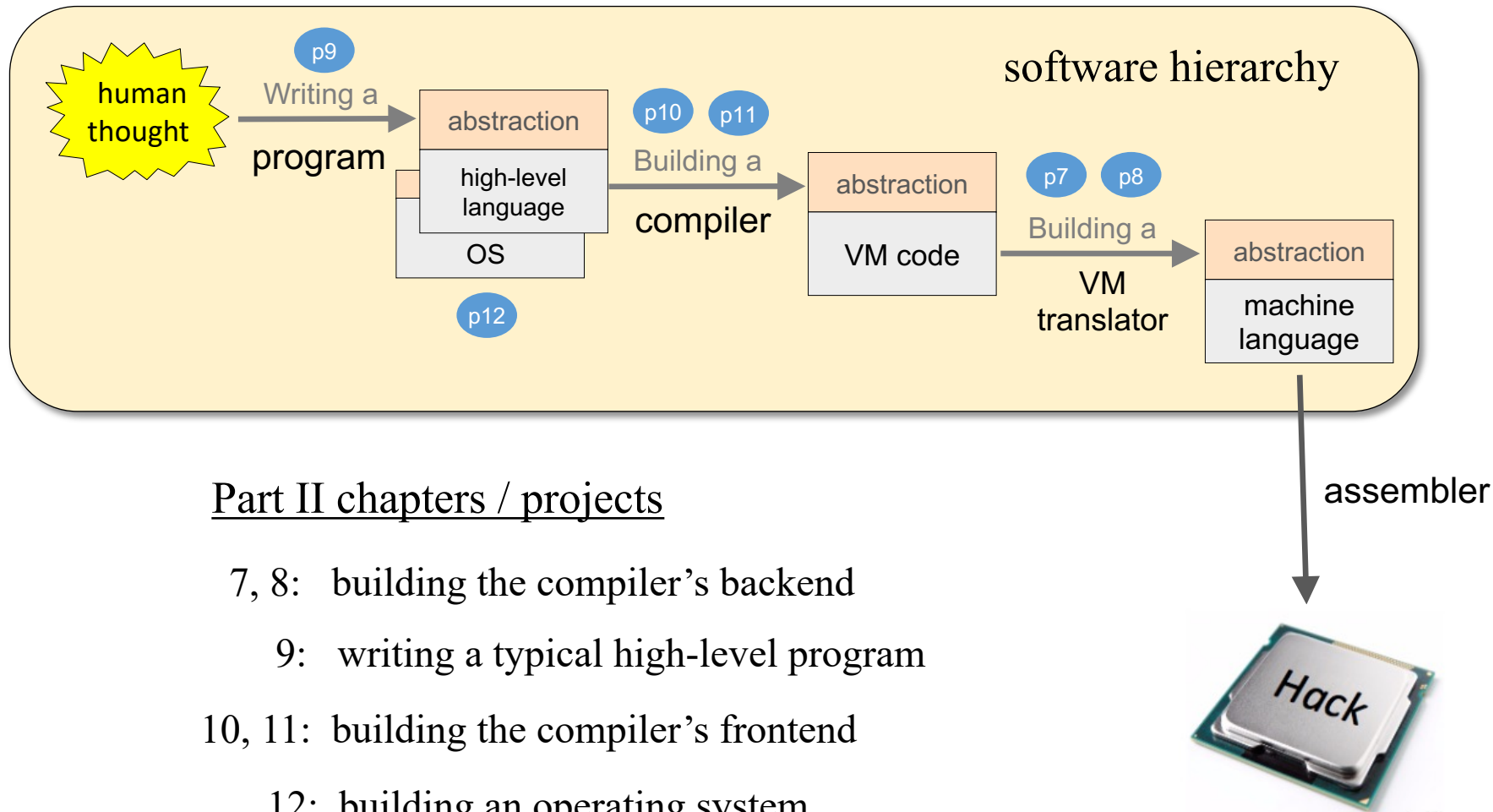
Cons

- Requires multiple VM translators, one for each target hardware platform
(But: VM translators are much simpler than compilers)
- Generates less efficient machine language code
(But: For numerous apps, the inefficiency delta is insignificant).

Compilation on the Hack/Jack platform

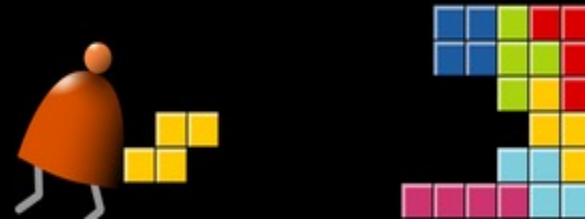


Nand to Tetris Roadmap: Part II



Part II chapters / projects

- 7, 8: building the compiler's backend
- 9: writing a typical high-level program
- 10, 11: building the compiler's frontend
- 12: building an operating system.



Chapter 7

Virtual Machine (Part I)

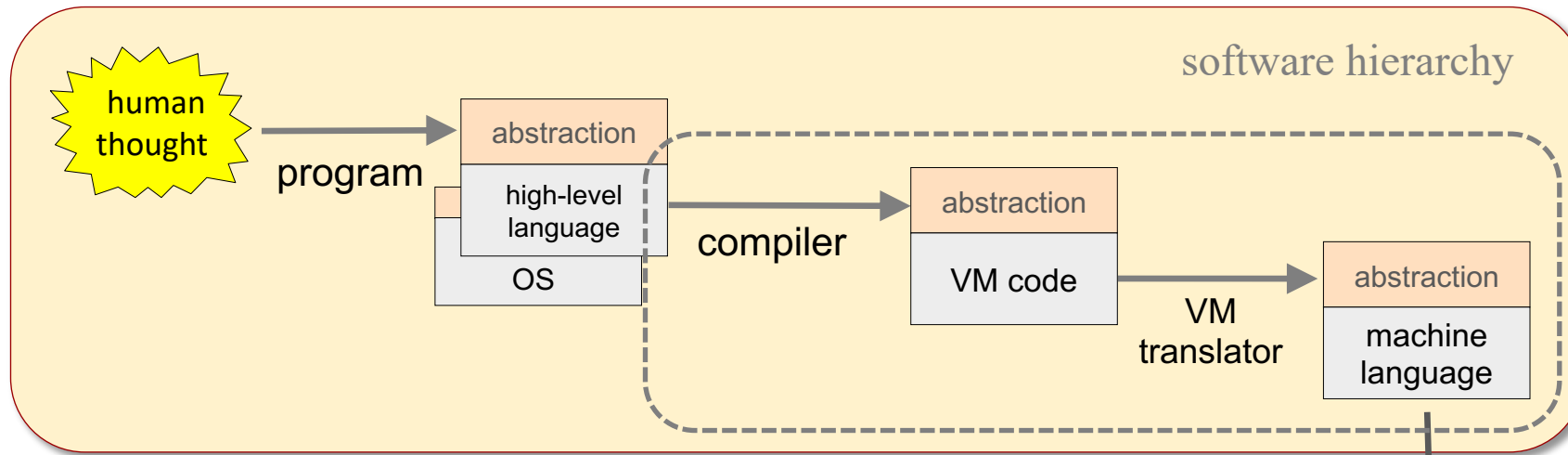
These slides support chapter 7 of the book

The Elements of Computing Systems

By Noam Nisan and Shimon Schocken

MIT Press, 2021

Nand to Tetris Roadmap: Part II



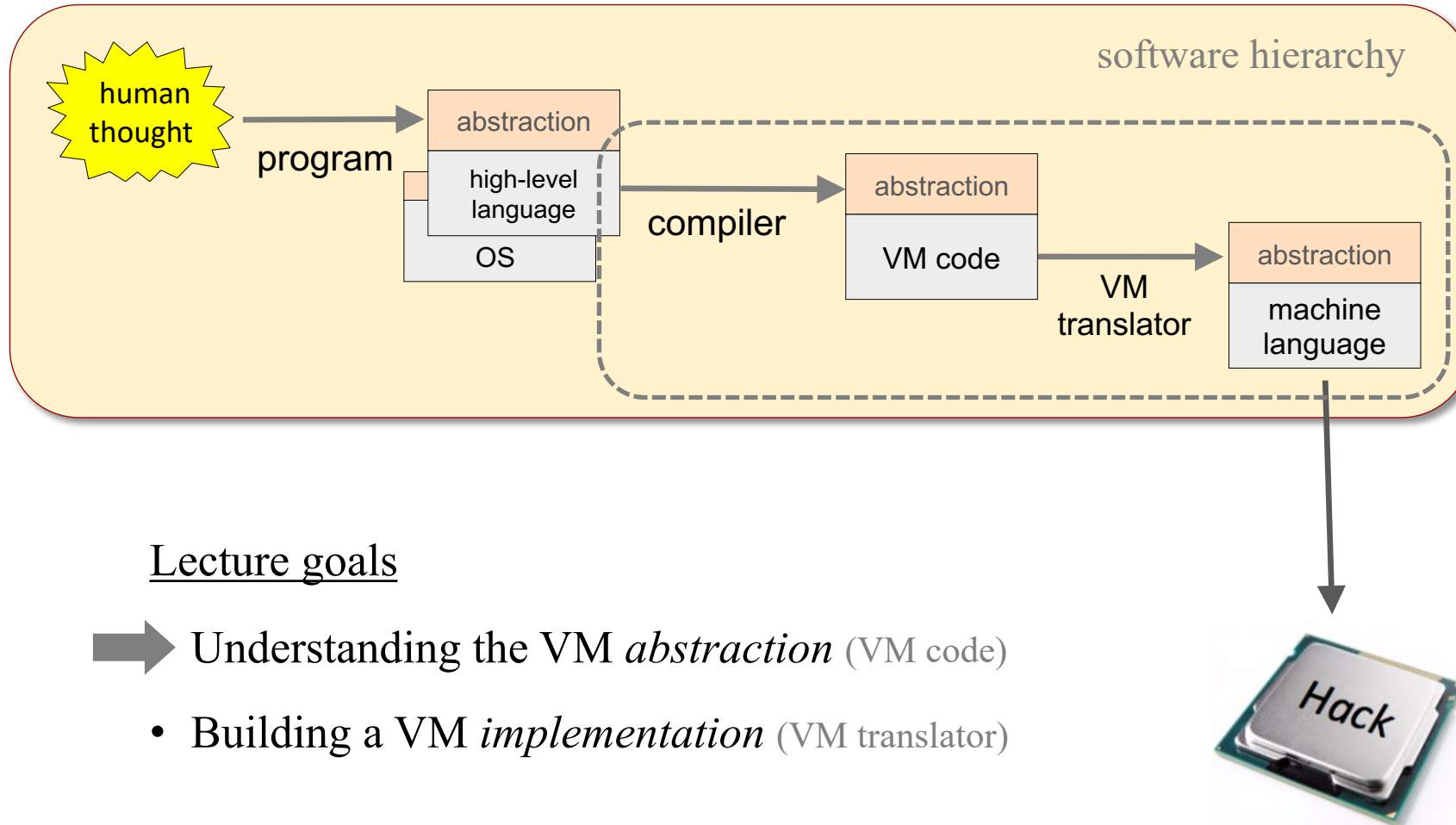
The VM code that the compiler generates is designed to operate on an *abstract stack machine*

The VM Translator translates this code into machine language

In other words: The VM translator realizes / implements the abstract stack machine on the host platform.



Nand to Tetris Roadmap: Part II



Lecture goals

- ➔ Understanding the VM *abstraction* (VM code)
 - Building a VM *implementation* (VM translator)

Stack

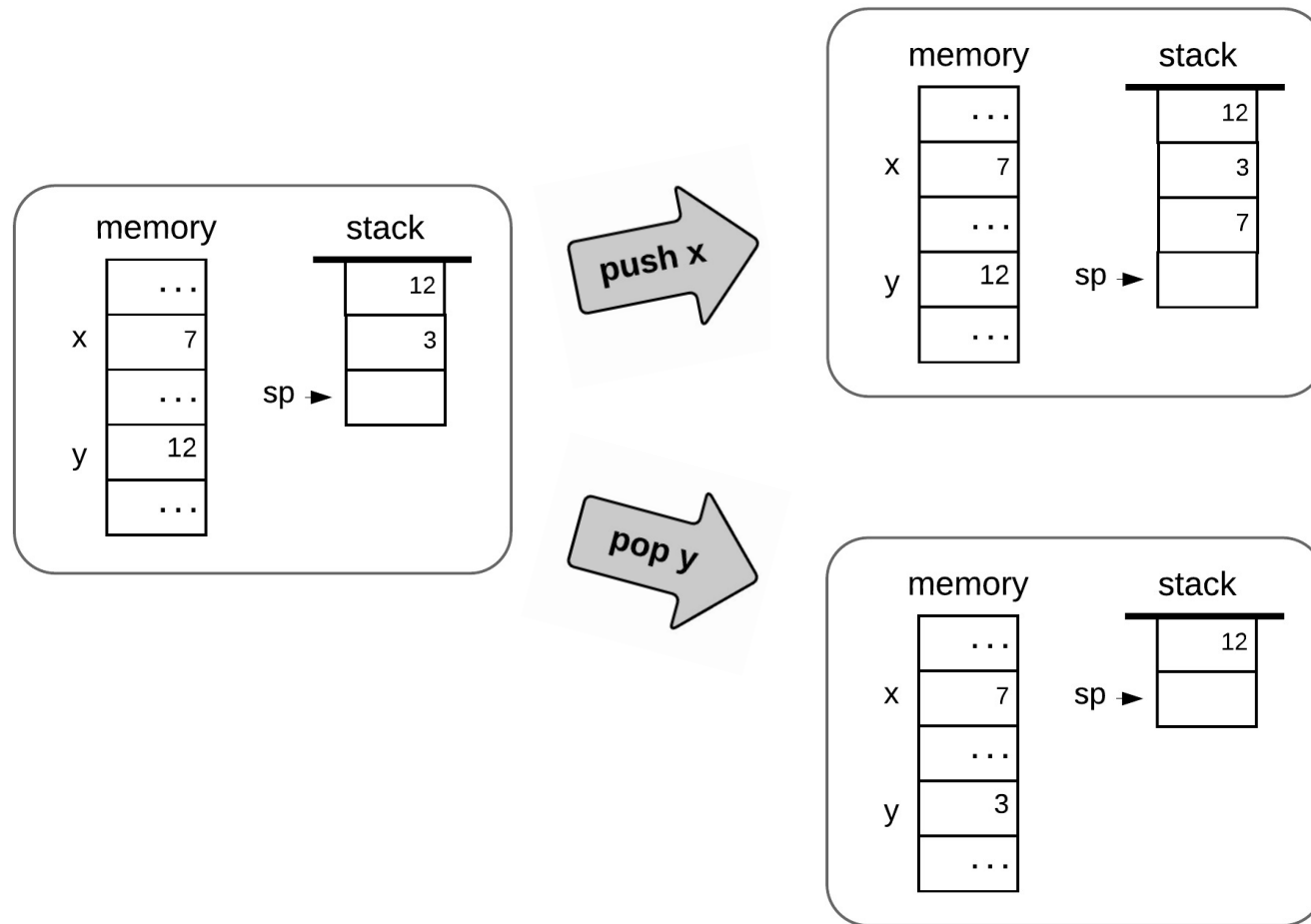


Basic operations

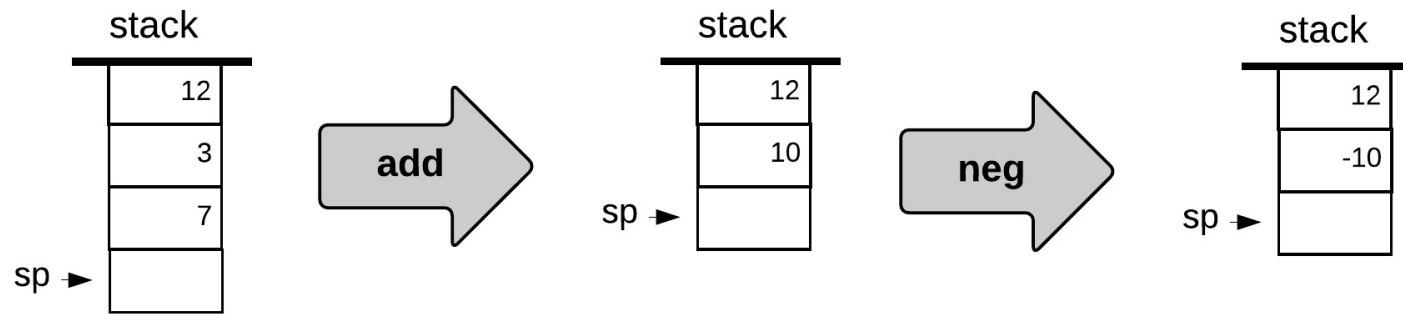
push: adds an element at the stack's top

pop: removes the top element

Stack

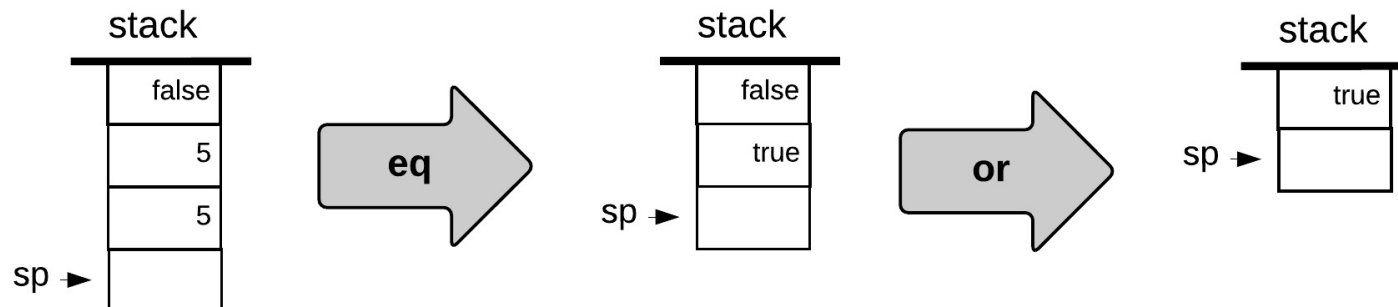


Stack arithmetic



Applying a function f (that has n arguments)

- pops n values (arguments) from the stack,
- Computes f on the values,
- Pushes the resulting value onto the stack.



Arithmetic operations

VM pseudocode (example)

```
// d = (2 - x) + (y + 9)
```

```
push 2
```

```
push x
```

```
sub
```

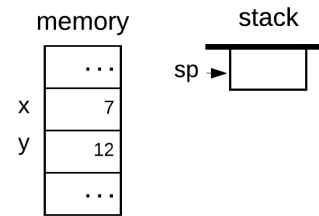
```
push y
```

```
push 9
```

```
add
```

```
add
```

```
pop d
```



Arithmetic operations

VM pseudocode (example)

```
// d = (2 - x) + (y + 9)
```

```
push 2
```

```
push x
```

```
sub
```

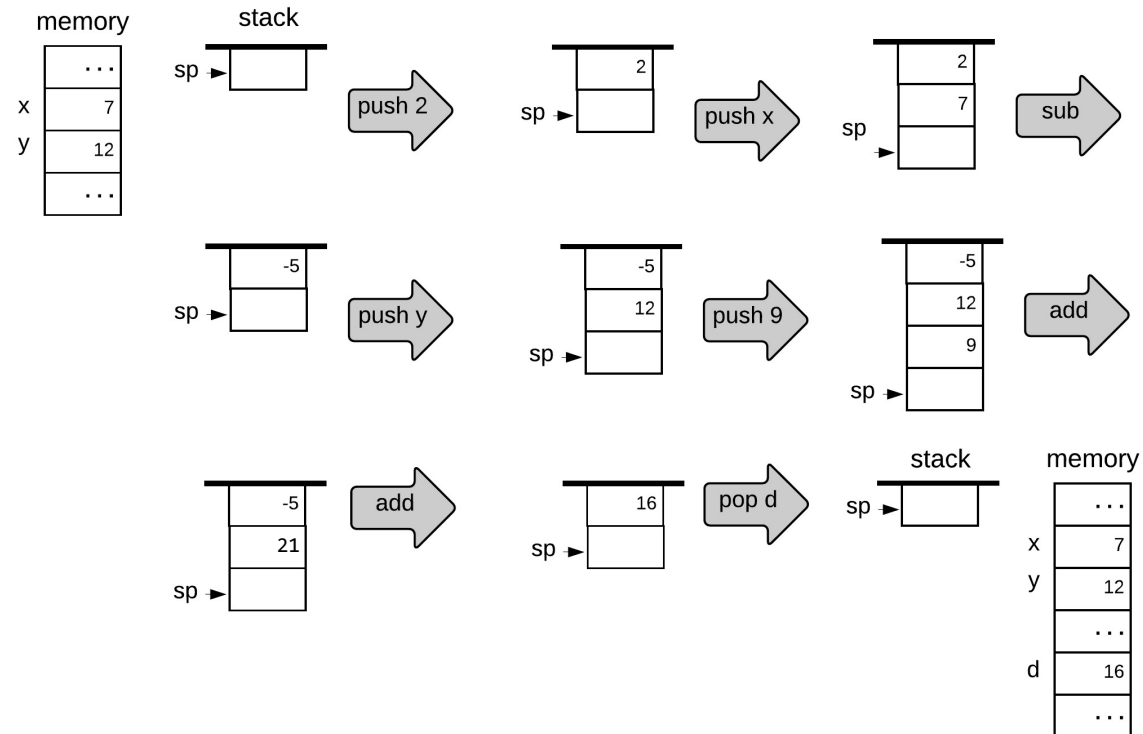
```
push y
```

```
push 9
```

```
add
```

```
add
```

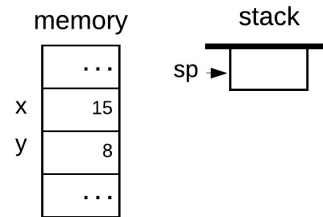
```
pop d
```



Logical operations

VM pseudocode (example)

```
// (x < 7) or (y == 8)
push x
push 7
lt
push y
push 8
eq
or
```



Logical operations

VM pseudocode (example)

```
// (x < 7) or (y == 8)
```

```
push x
```

```
push 7
```

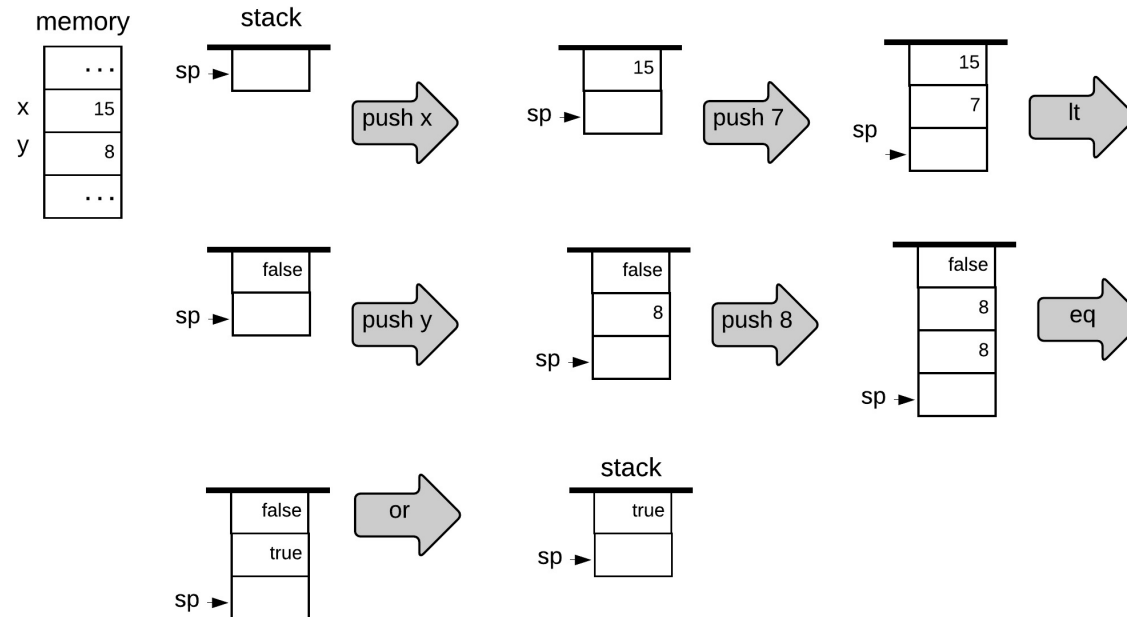
```
lt
```

```
push y
```

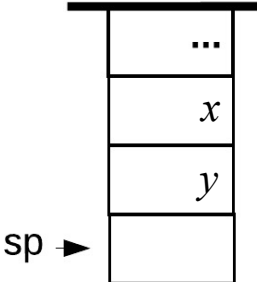
```
push 8
```

```
eq
```

```
or
```



Arithmetic / Logical VM operations

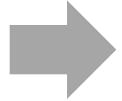
Command	Return value	Return value	stack
add	$x + y$	integer	 sp ➔
sub	$x - y$	integer	
neg	$-y$	integer	
eq	$x == y$	boolean	
gt	$x > y$	boolean	
lt	$x < y$	boolean	
and	$x \text{ And } y$	boolean	
or	$x \text{ Or } y$	boolean	
not	Not x	boolean	

The big picture: Compilation

VM code is typically written by compilers;

Every high-level arithmetic-logical expression can be translated into a sequence of operations on a stack, using these VM commands.

The VM language (*abstraction*)



Push / pop commands

- `push segment i`
- `pop segment i`

Branching commands

- `label label`
- `goto label`
- `if-goto label`



Arithmetic / Logical commands

- `add, sub, neg`
- `eq, gt, lt`
- `and, or, not`

Function commands

- `Function functionName nVars`
- `Call functionName nArgs`
- `return`

The big picture: Compilation

Source code (Jack)

```
class Foo {  
    static int s1, s2;  
    function int bar (int x, int y) {  
        var int a, b, c;  
        ...  
        let c = s1 + y;  
        ...  
    }  
}
```

each variable has a *kind*:
local, argument, static, ...

compiler

Compiled VM code

```
...  
...  
...  
...  
push static 0  
push argument 1  
add  
pop local 2  
...
```

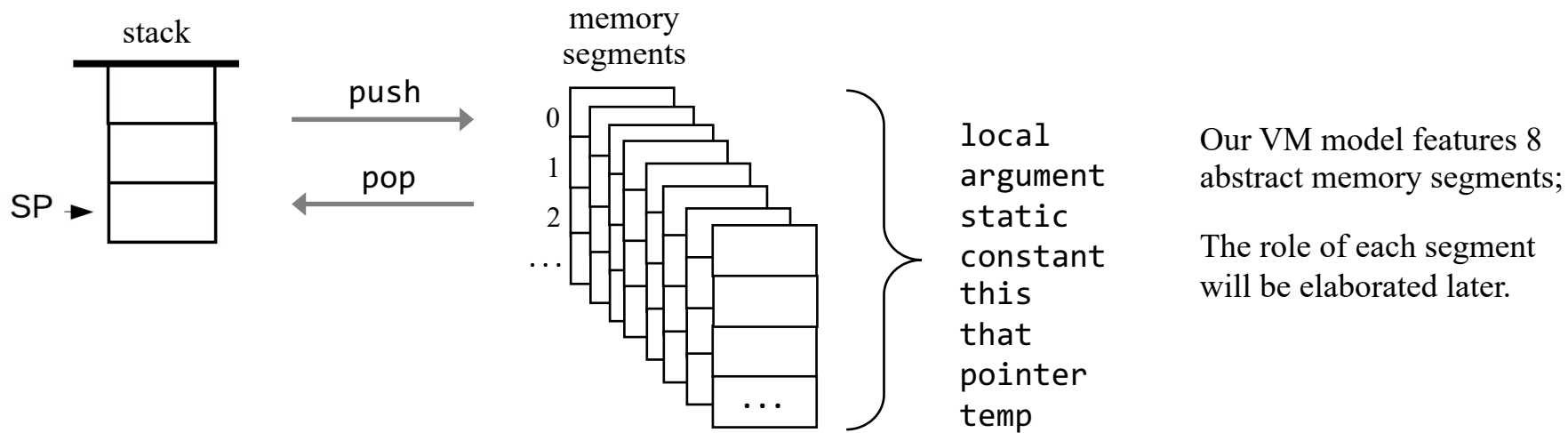
local		argument		static	
0		0	x	0	s1
1	b	1	y	1	s2
2	c	2		2	
3		3		3	...
...	...	...	...	...	...

Abstract memory segments

The compiler represents each symbolic variable as an entry in an abstract memory segment

The compiler translates each high-level statement into a sequence of VM commands.

Memory segments



All segments are accessed the same way:

`push / pop segment i`

where i is a non-negative integer.

The VM language: Implementation

Push / pop commands

- `push segment i`
- `pop segment i`

Arithmetic / Logical commands

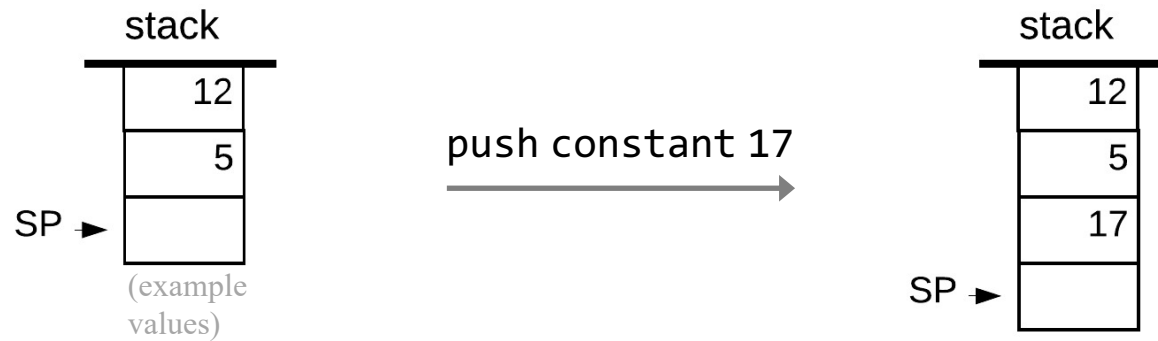
- `add, sub, neg`
- `eq, gt, lt`
- `and, or, not`

How can we make these commands “work”?

- We’ll develop a *VM translator*:
- A program that translates each VM command into machine language instructions

Implementing push constant i

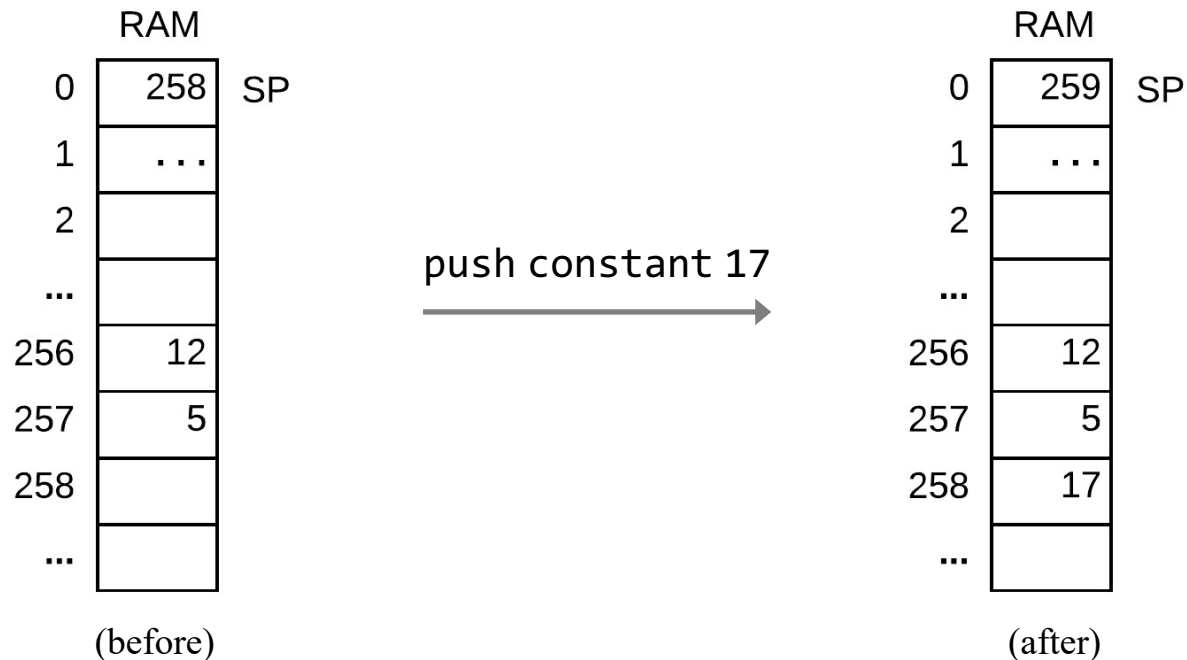
Abstraction



Implementation

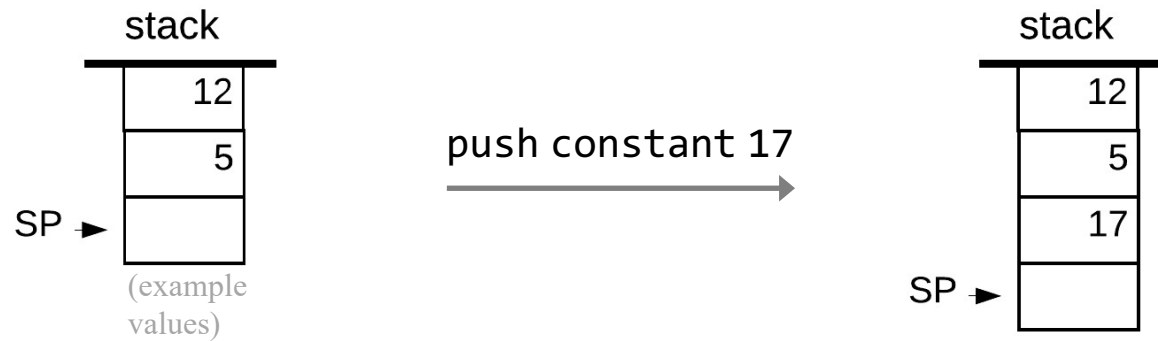
The *stack* is stored in the RAM, from address 256 onward

The *stack pointer* is stored in RAM[0]



Implementing push constant i

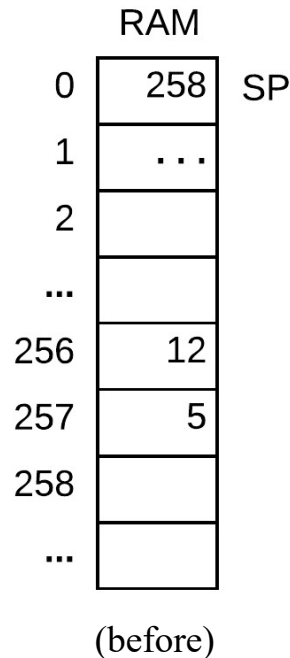
Abstraction



Implementation

The *stack* is stored in the RAM, from address 256 onward

The *stack pointer* is stored in RAM[0]

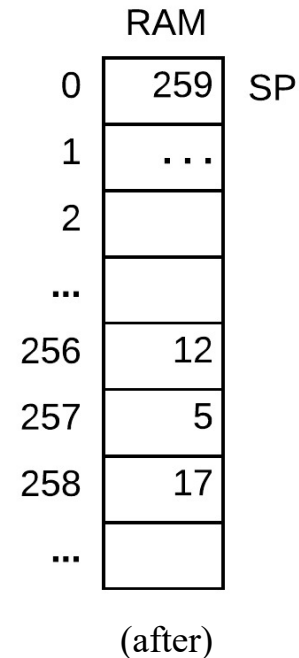


Pseudocode

```
// push constant 17
RAM[ SP ] = 17
SP++
```

Hack assembly

```
// D = 17
@17
D=A
// RAM[ SP ] = D
@SP
A=M
M=D
// SP++
@SP
M=M+1
```



Implementing push constant i

Abstraction

VM code

```
push constant  $i$ 
```

VM translator

Implementation

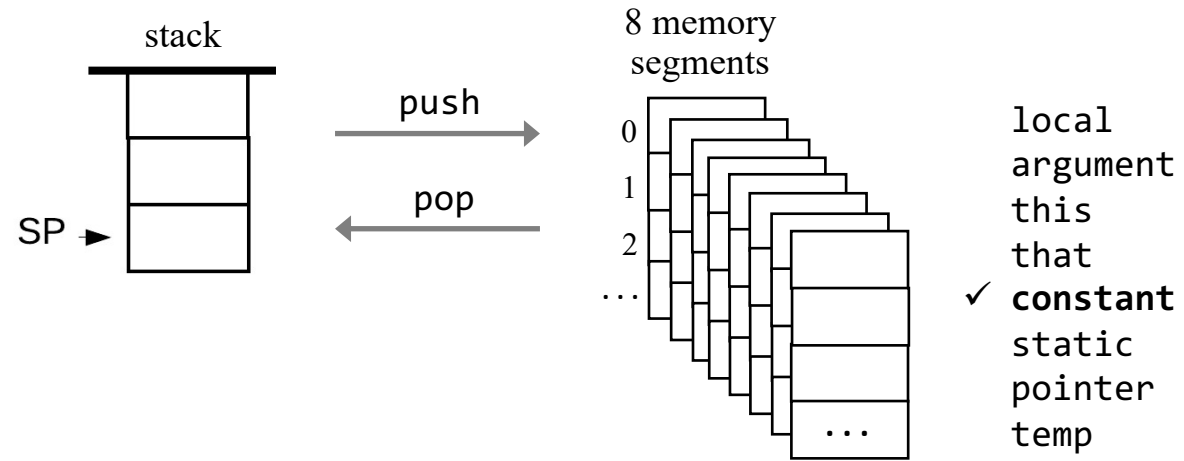
Assembly code

```
// D =  $i$   
@ $i$   
D=A  
// RAM[SP] = D  
@SP  
A=M  
M=D  
// SP++  
@SP  
M=M+1
```

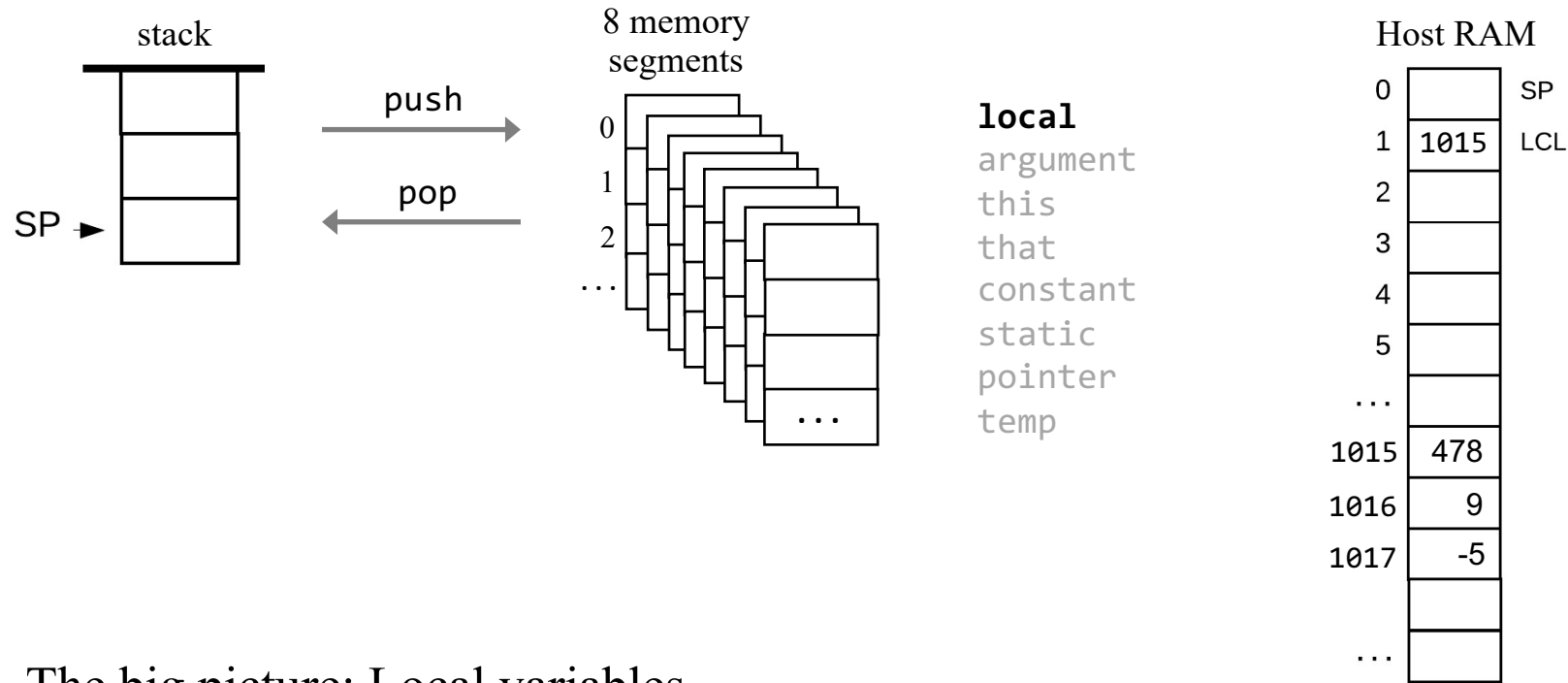
Notes

1. constant is not a “real segment”
(used for VM syntax consistency)
2. There is no pop constant i command.

Implementing push/pop



Implementing push/pop local i



The big picture: Local variables

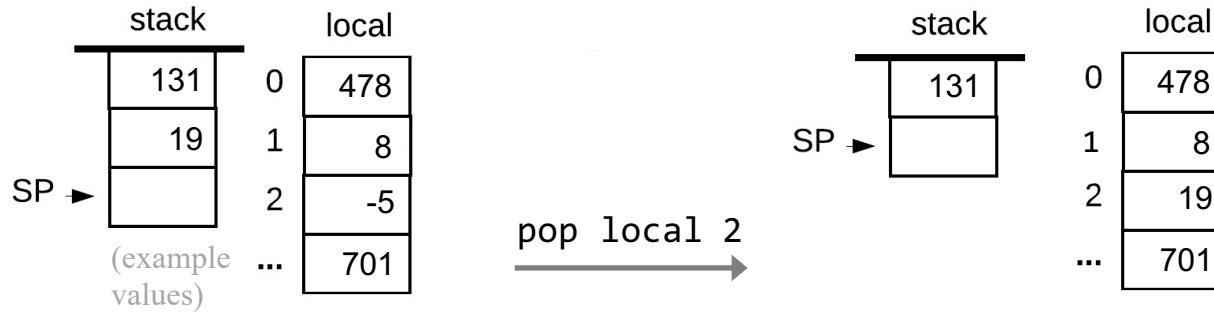
When the compiler translates a high-level *method*, it allocates a RAM block for representing the method's *local variables*

Abstraction: local 0, local 1, local 2, ...

Implementation: A RAM block whose base address is kept in a pointer named **LCL**

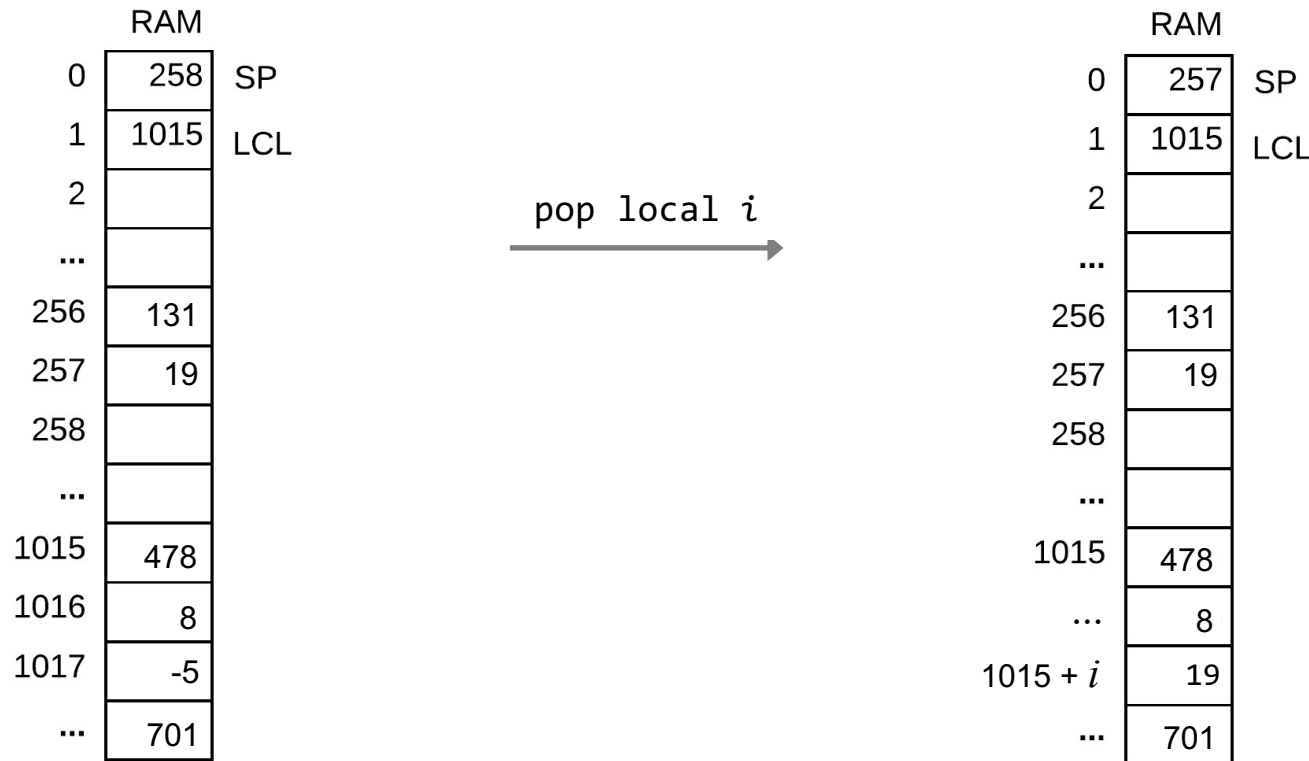
Implementing push/pop local i

Abstraction



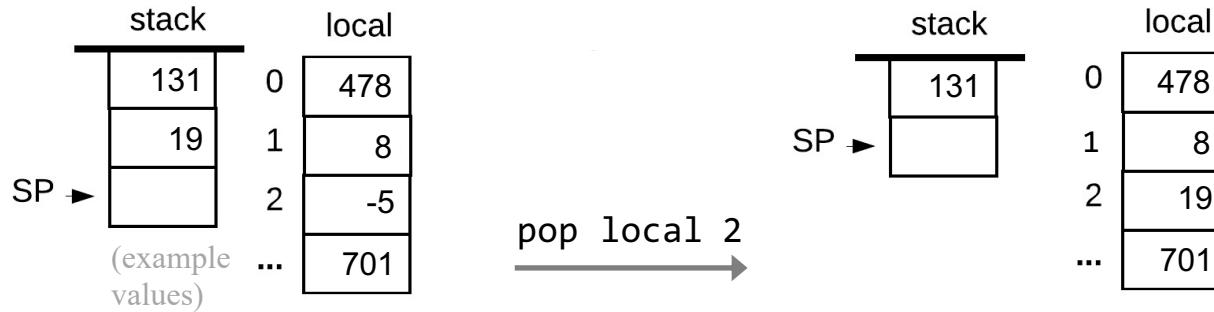
Implementation

Here, the base address happens to be 1015 (arbitrary example)



Implementing push/pop local i

Abstraction



Implementation

Here, the base address happens to be 1015 (arbitrary example)

RAM		
0	258	SP
1	1015	LCL
2		
...		
256	131	
257	19	
258		
...		
1015	478	
1016	8	
1017	-5	
...	701	

Pseudocode

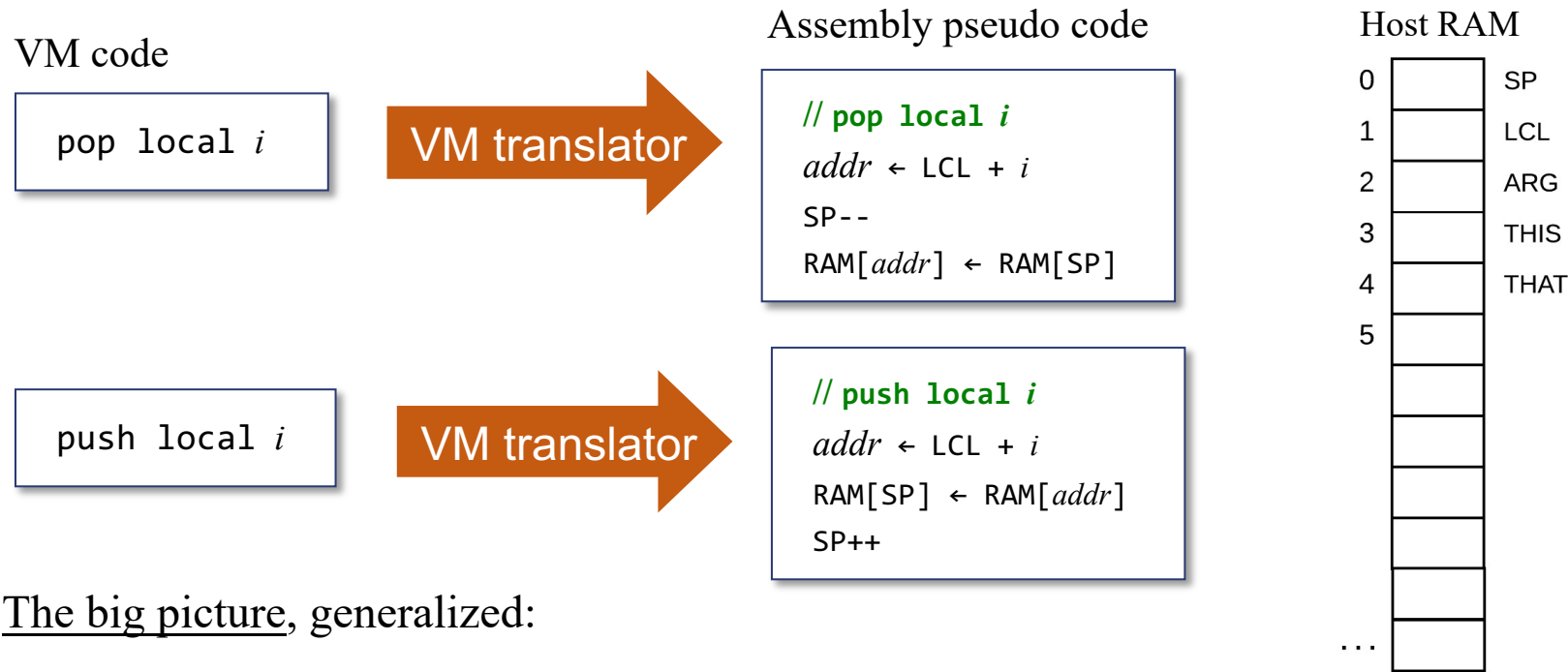
```
// pop local  $i$   
 $addr \leftarrow LCL + i$   
 $SP--$   
 $RAM[addr] \leftarrow RAM[SP]$ 
```

Hack assembly

You figure it out!

RAM		
0	257	SP
1	1015	LCL
2		
...		
256	131	
257	19	
258		
...		
1015	478	
...	8	
$1015 + i$	19	
...	701	

Implementing push/pop local *i*



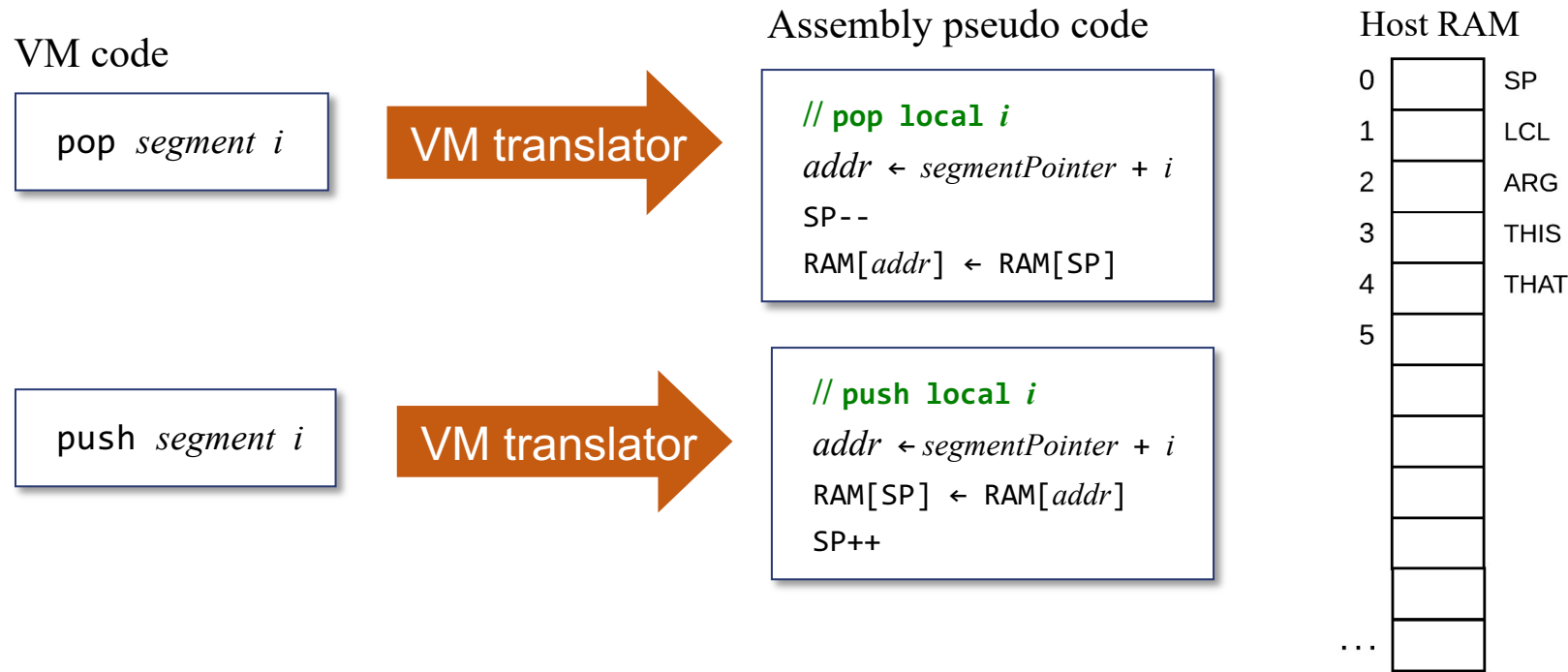
The big picture, generalized:

When the compiler compiles a high-level method, it represents...

- the method's *local variables* in the virtual segment `local`
- the method's *arguments* in the virtual segment `argument`
- the *fields* of the current object in the virtual segment `this`
- the *elements* of the current array in the virtual segment `that`

All abstracted and
implemented exactly
the same way

Implementing `push/pop {local | argument | this | that} i`

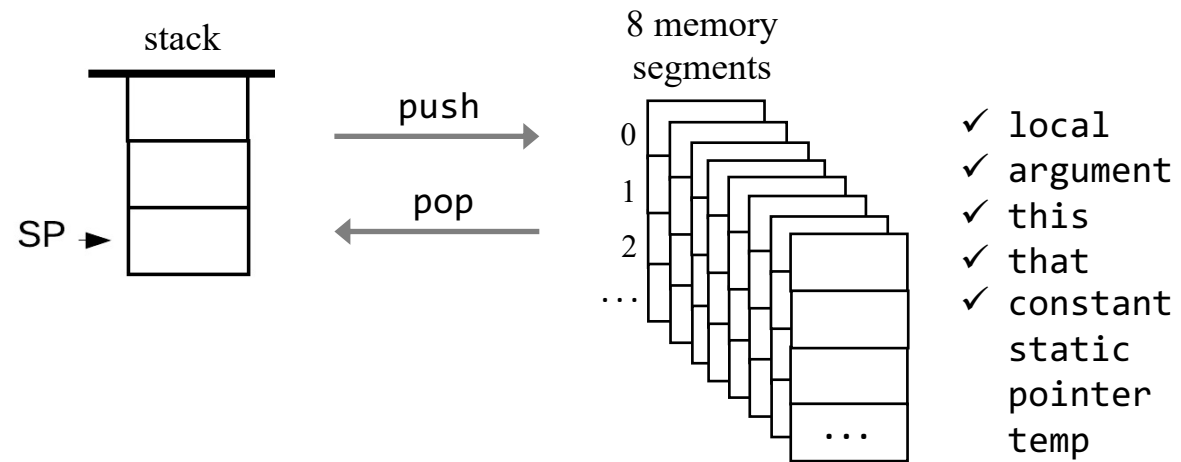


where `segment` is local, argument, this, that

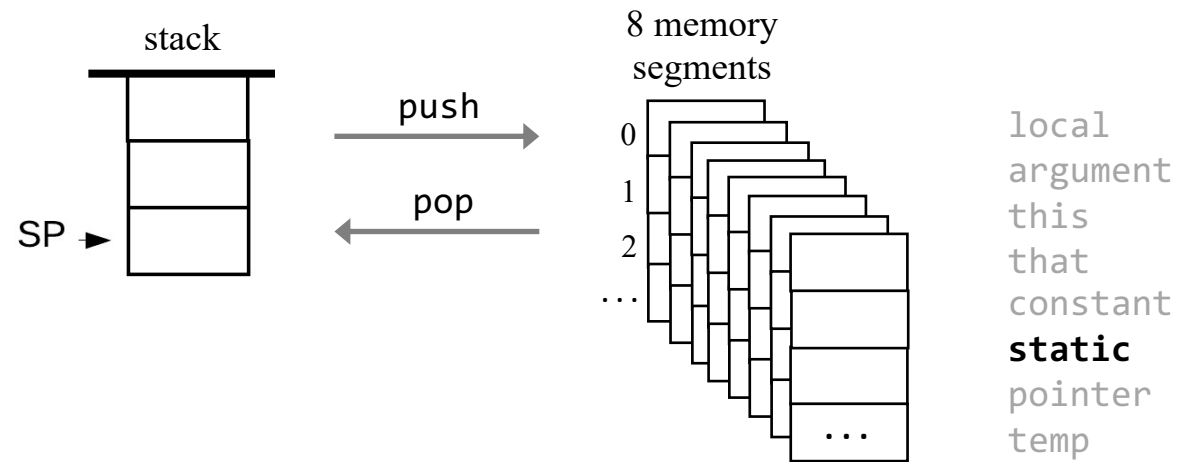
and `segmentPointer` is LCL, ARG, THIS, THAT

All abstracted and
implemented exactly
the same way

Implementing push/pop



Implementing `push/pop static i`



Implementing push/pop static *i*

Jack source code

```
class Bar {  
    ...  
    class Foo {  
        static int s1, s2;  
        ...  
        function int bar (int x, int y) {  
            var int a, b, c;  
            ...  
            let c = s1 + y;  
            ...  
        }  
    }  
}
```

Bar.jack

Foo.jack

compiler

Compiled VM code

Bar.vm

Foo.vm

...
...
...
...
...
...
...

The Big Picture: Static variables

- A Jack program is a collection of one or more Jack classes
- Each class is a separate compilation unit
- Each class file `Foo.jack` is compiled into a VM file named `Foo.vm`
- The compiler maps the class-level *static variables* of all the classes on one VM segment named `static`

Implementing `push/pop static i`

Implementation

The static segment is stored in a fixed RAM block, starting at address 16;

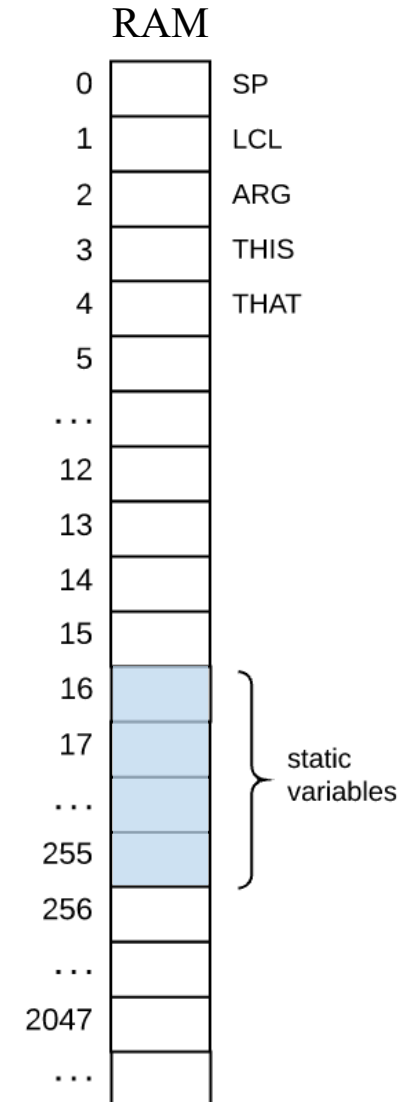
When translating a VM file named `Foo.vm`:

To `push/pop static i`, generate assembly code that executes
`push/pop Foo.i`

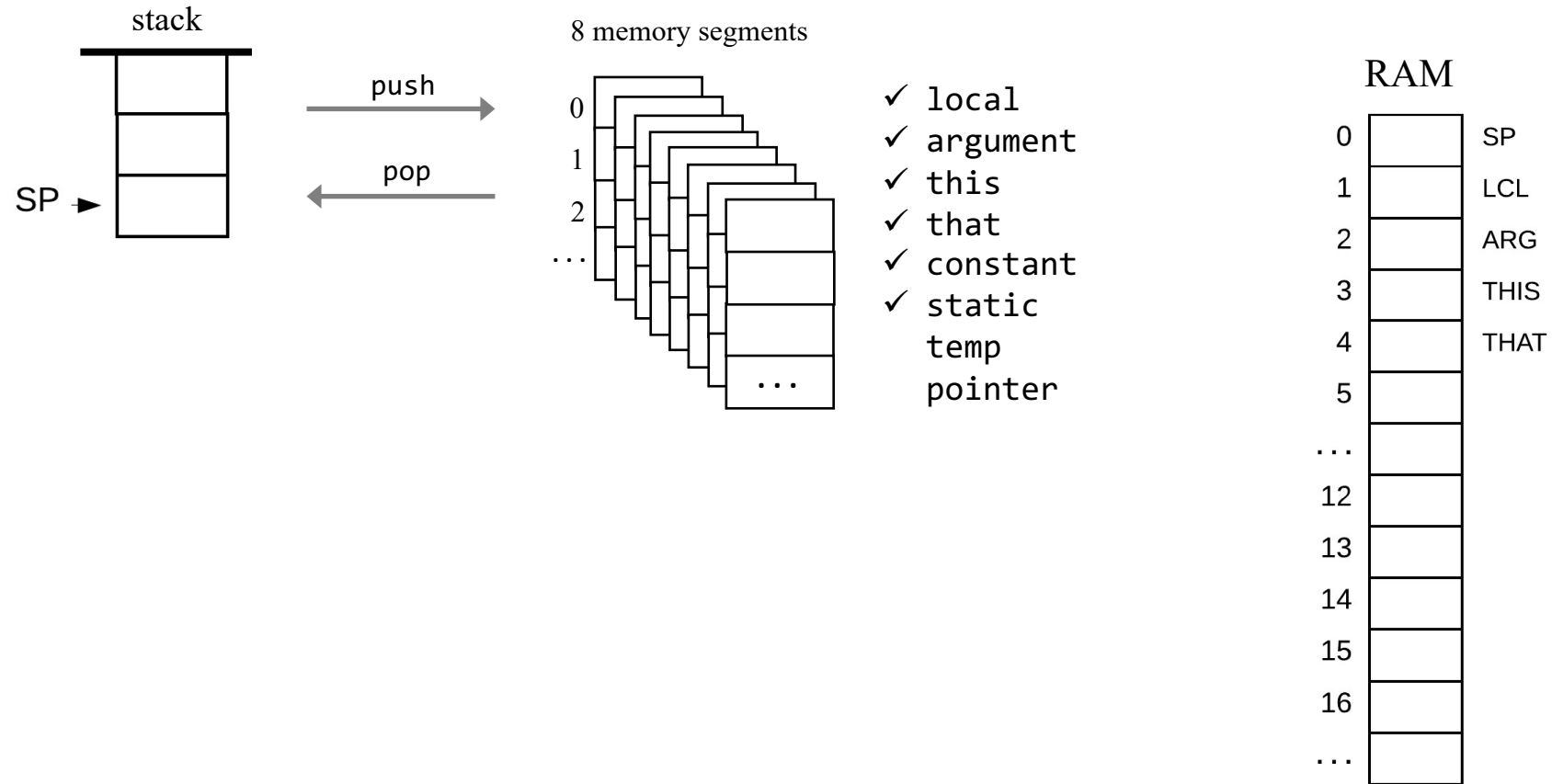
Explanation

The Hack assembler will subsequently map every symbol `Foo.i` in the generated assembly code onto `RAM[16]`, `RAM[17]`, ..., `RAM[255]` (exactly what we want...)

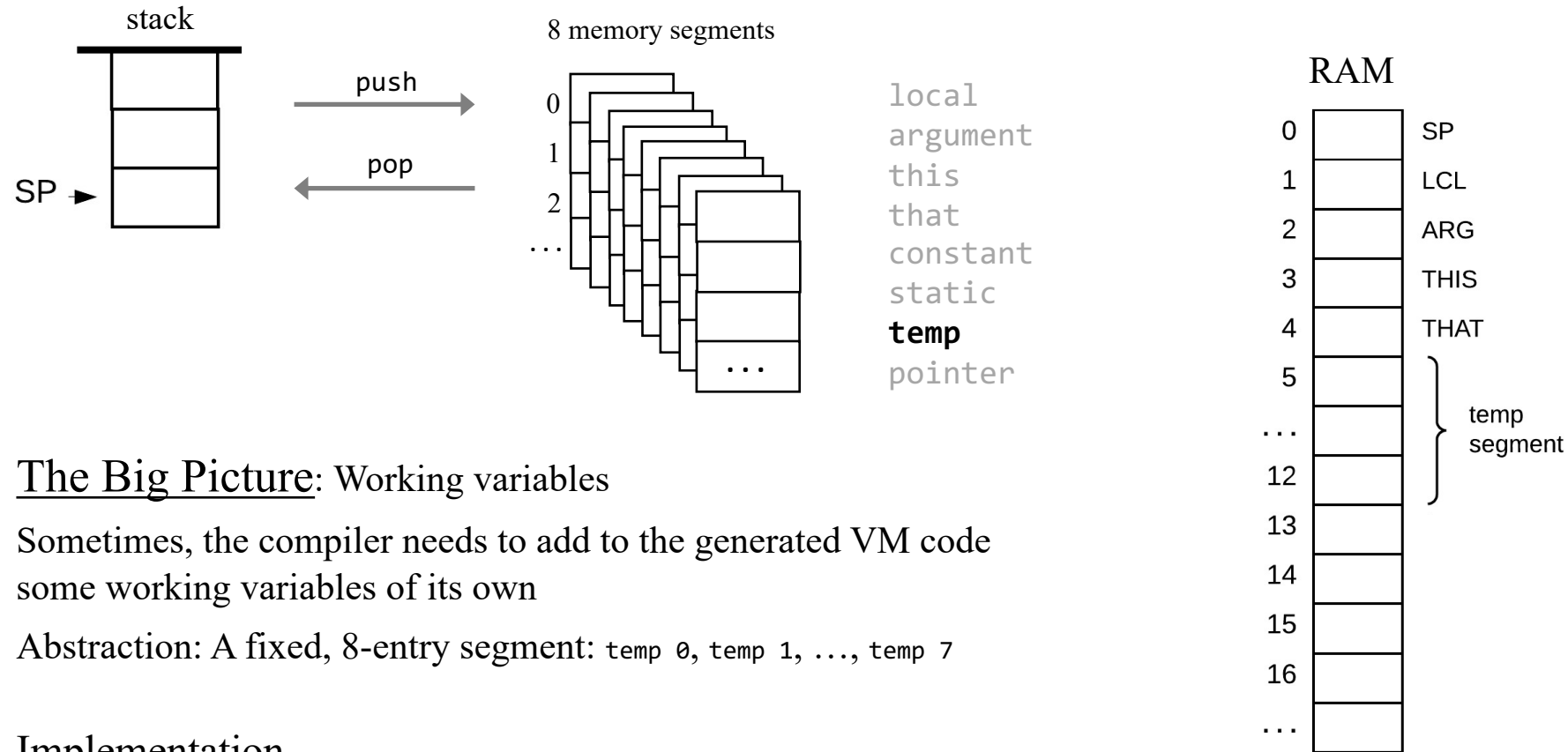
Note: This is quite different from *local variables*, which are created when a function starts running, and recycled when the function terminates (to be discussed in the next lecture).



Implementing push/pop



Implementing push/pop temp i



The Big Picture: Working variables

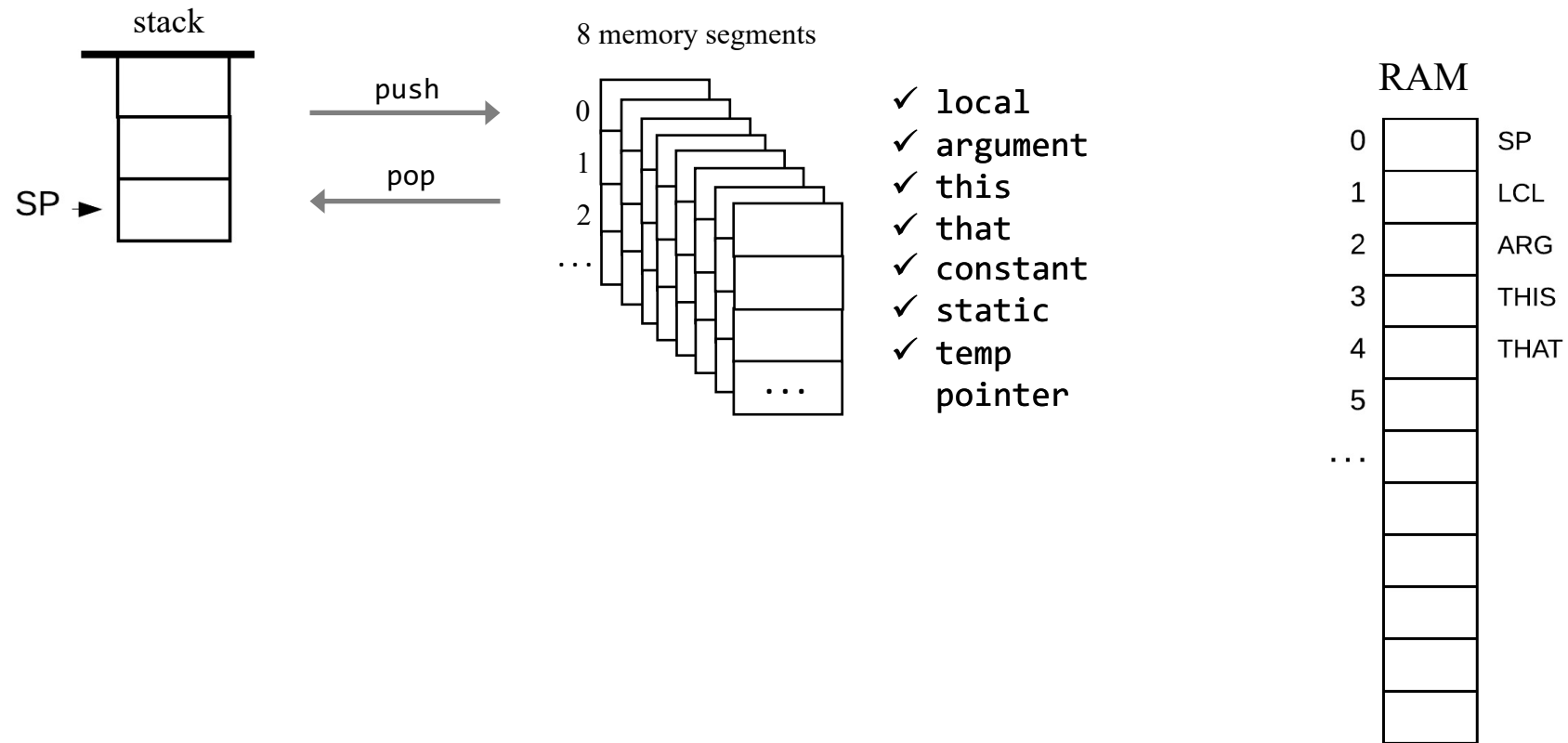
Sometimes, the compiler needs to add to the generated VM code some working variables of its own

Abstraction: A fixed, 8-entry segment: temp 0, temp 1, ..., temp 7

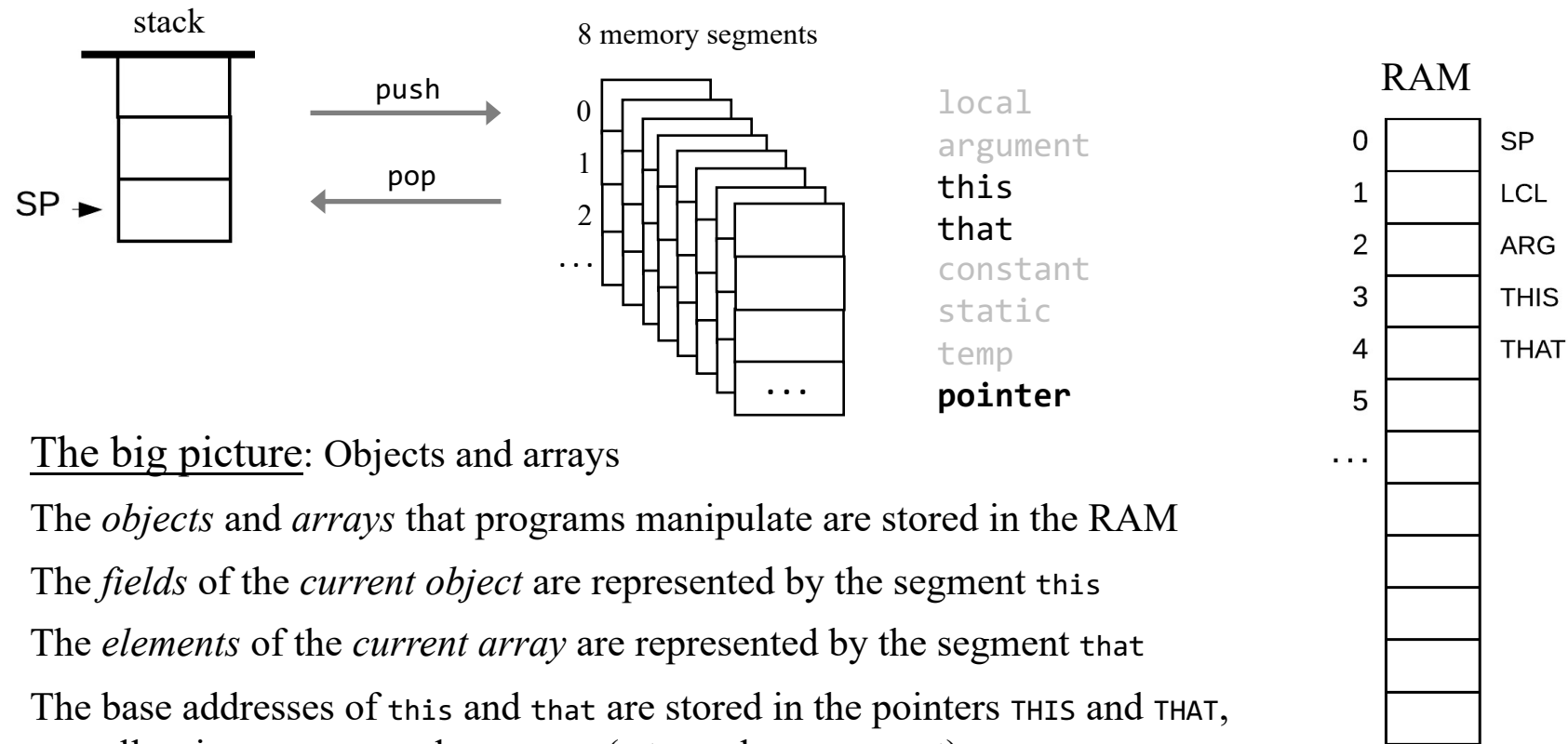
Implementation

- The temp segment is stored in a fixed RAM block, from address 5 to 12
- To push/pop temp i , generate assembly code that executes
push/pop RAM[5 + i]

Implementing push/pop



Implementing push/pop pointer i



The big picture: Objects and arrays

The *objects* and *arrays* that programs manipulate are stored in the RAM

The *fields* of the *current object* are represented by the segment *this*

The *elements* of the *current array* are represented by the segment *that*

The base addresses of *this* and *that* are stored in the pointers *THIS* and *THAT*, as well as in pointer 0 and pointer 1 (a two-place segment)

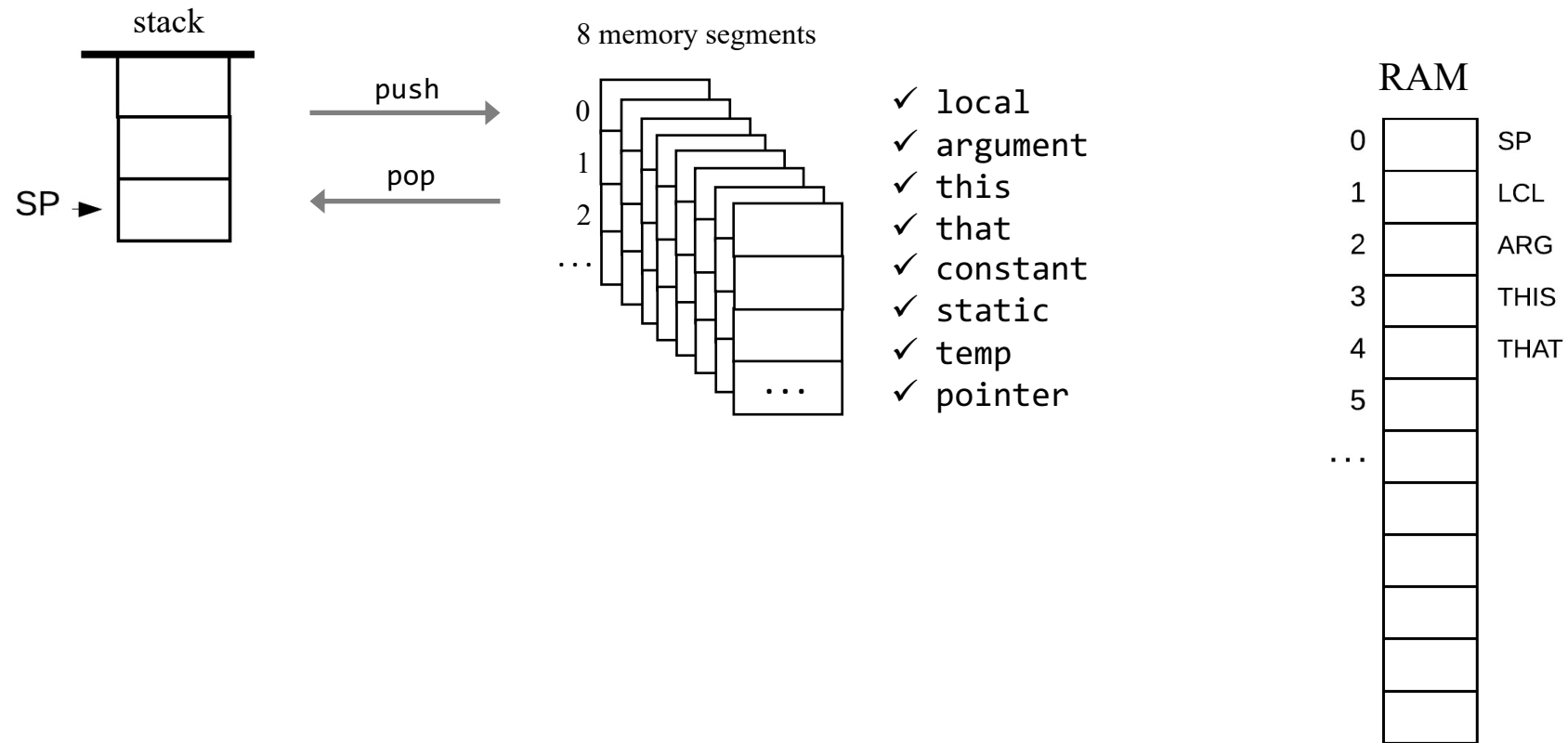
Implementation

To push/pop pointer 0, generate assembly code that executes: push/pop THIS

To push/pop pointer 1, generate assembly code that executes: push/pop THAT

} The pointer segment will come to play when we'll build the compiler, stay tuned

Implementing push/pop



The VM language: Implementation



Push / pop commands

- `push segment i`
- `pop segment i`



Arithmetic / Logical commands

- `add, sub, neg`
- `eq, gt, lt`
- `and, or, not`

Implementing the arithmetic-logical commands

Command	Return value	Return value
add	$x + y$	integer
sub	$x - y$	integer
neg	$-y$	integer
eq	$x == y$	boolean
gt	$x > y$	boolean
lt	$x < y$	boolean
and	$x \text{ And } y$	boolean
or	$x \text{ Or } y$	boolean
not	Not x	boolean

Abstraction

Each arithmetic/logical command pops one or two values from the stack, computes one of these functions on these values, and pushes the computed value onto the stack

Implementation

We already know how to implement push / pop;

The rest is easy.

The VM language

✓ Push / pop commands

- push *segment i*
- pop *segment i*

✓ Arithmetic / Logical commands

- add, sub , neg
- eq , gt , lt
- and, or , not



This
lecture

Branching commands

- label *label*
- goto *label*
- if-goto *label*

Function commands

- Function *functionName nVars*
- Call *functionName nArgs*
- return



Next
lecture

Virtual Machine I: Lecture plan

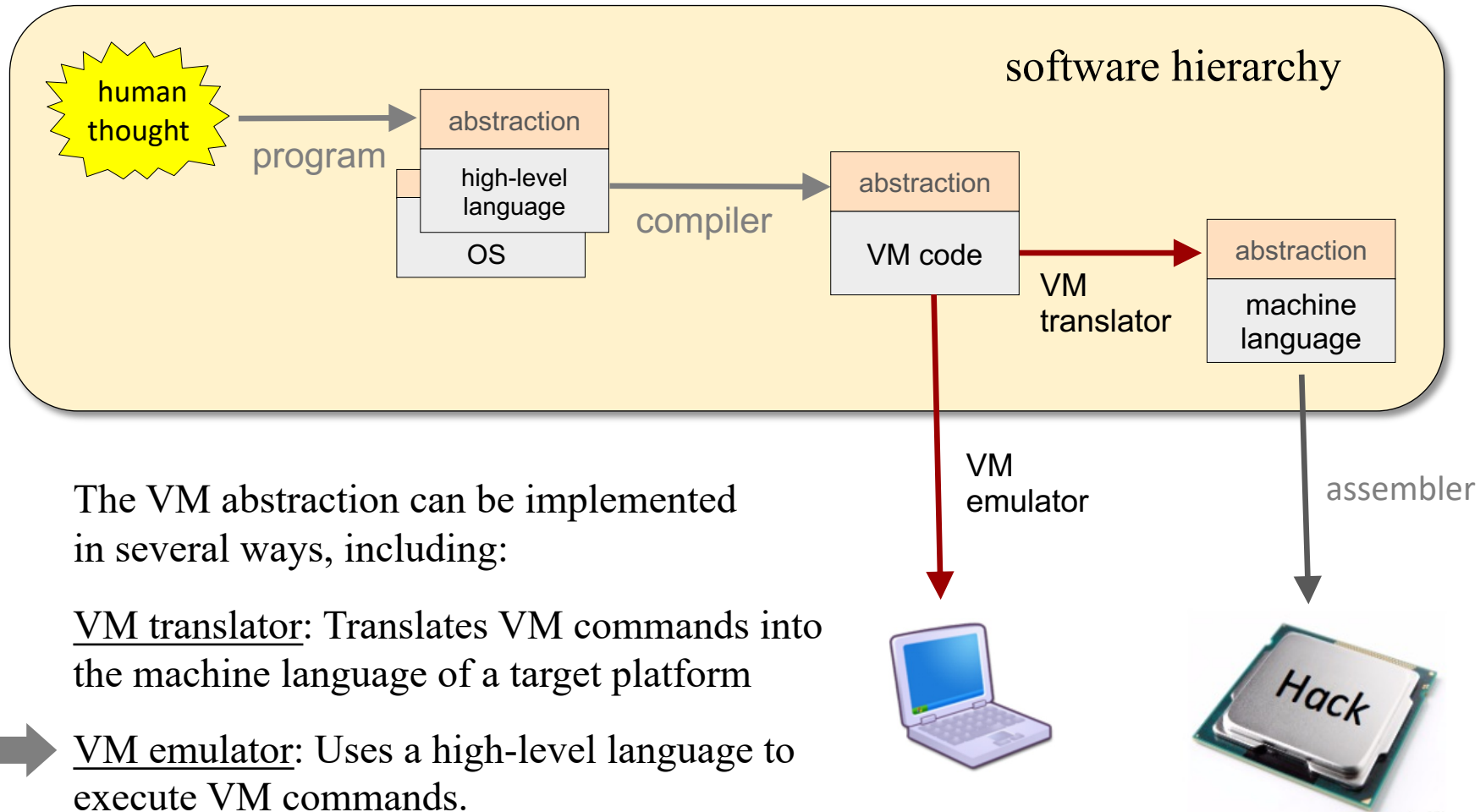
- Overview
- VM Language: Abstraction
- VM Language: Implementation



VM Emulator

- Standard Mapping
- VM Translator
- Project 7

VM implementations



Our VM emulator

The screenshot displays a VM emulator interface with the following components:

- Menu Bar:** File, View, Run, Help.
- Toolbar:** Includes icons for file operations, execution (single step, step over, step into, run), and animation controls (Slow, Fast). It also features dropdowns for 'Animate:' (set to Program flow), 'View:' (set to Scr...), and 'Format:' (set to D...).
- Program List:** A list of 15 instructions. Instruction 10, 'pop that 5', is highlighted in yellow.
- Static Memory:** A table with 5 slots, all containing 0.
- Local Memory:** A table with 5 slots. Slot 0 contains 10, while slots 1-4 contain 0.
- Argument Memory:** A table with 5 slots. Slot 0 contains 0, slot 1 contains 21, slot 2 contains 22, and slots 3-4 contain 0.
- This Memory:** A table with 6 slots. Slots 0-5 all contain 0.
- That Memory:** A table with 2 slots, both containing 0.
- Temp Memory:** A table with 1 slot, containing 0.
- Stack:** A vertical list showing values 42 and 45. The value 45 is highlighted in blue.
- Call Stack:** An empty list.
- Global Stack:** A table of memory addresses and values. Address 258 is highlighted in yellow and contains the value 0.
- RAM:** A table of memory addresses and values. Address 0 is highlighted in yellow and contains the value 258.
- Visual Display:** A large window showing a 3D rendering of a checkered ball on a surface.

Our VM emulator

The screenshot shows the Nand2Tetris VM emulator interface. A blue box highlights the top toolbar, which includes menu items (File, View, Run, Help), navigation icons, and execution controls (Slow, Fast, Animate: Program flow). A yellow callout points to this area with the text "execution controls".

On the left, the "Program" list shows instructions 0-14. Instruction 10, "pop that 5", is highlighted in yellow. A yellow callout points to it with the text "VM code". Below the program list is the "Stack" window showing values 42 and 45, with a yellow callout pointing to it labeled "stack". At the bottom left is the "Call Stack" window.

A large blue box highlights the central "memory segments" area, which includes Static, Local, Argument, This, That, and Temp registers. A yellow callout points to this area with the text "memory segments".

To the right of the memory segments is a large "Multi-purpose pane" (yellow callout) which can display program output, test scripts, output files, or compare files. Currently, it shows a checkered ball.

At the bottom right are the "Global Stack" and "RAM" windows. The RAM window shows memory addresses 0 to 14 with values, with address 0 highlighted in yellow.

Emulating a VM program

BasicTest.vm (example)

```
push constant 10
pop local 0
pop local 0
push constant 21
push constant 22
pop argument 2
pop argument 1
push constant 36
pop this 6
...
```

abstraction

implementation

Things to watch for:

- Stack state
- Memory segments states

How the stack and memory segments are realized on the host platform (Hack RAM)

Emulating a VM program: Testing

BasicTest.vm (example)

```
push constant 10
pop local 0
push constant 21
push constant 22
pop argument 2
pop argument 1
push constant 36
pop this 6
...
```

BasicTestVME.tst

```
load BasicTest.vm,
output-file BasicTest.out,
compare-to BasicTest.cmp,
```

The *progNameVME.tst* scripts enable experimenting with the VM test programs in the VM emulator

- There is no need to study the details of the test scripts
- But, it helps to understand the general testing logic.

Emulating a VM program: Testing

BasicTest.vm (example)

```
push constant 10
pop local 0
push constant 21
push constant 22
pop argument 2
pop argument 1
push constant 36
pop this 6
...
```

BasicTestVME.tst

```
load BasicTest.vm,
output-file BasicTest.out,
compare-to BasicTest.cmp,
// Specifies selected values to output, and their format:
output-list RAM[256]%D1.6.1 RAM[300]%D1.6.1 ...
```

Instructs to output the *stack base*, and selected values from selected memory segments (as they are stored in the host RAM)

Emulating a VM program: Testing

BasicTest.vm (example)

```
push constant 10
pop local 0
push constant 21
push constant 22
pop argument 2
pop argument 1
push constant 36
pop this 6
...
```

BasicTestVME.tst

```
load BasicTest.vm,
output-file BasicTest.out,
compare-to BasicTest.cmp,
// Specifies selected values to output, and their format:
output-list RAM[256]%D1.6.1 RAM[300]%D1.6.1 ...
```

Executes the VM program

```
repeat 25 {           // BasicTest.vm requires 25 VM steps
  vmstep;
}
// Outputs the values specified by the output-list
// (stack base + selected values from selected memory segments)
output;
```

Emulating a VM program: Testing

BasicTest.vm (example)

```
push constant 10
pop local 0
push constant 21
push constant 22
pop argument 2
pop argument 1
push constant 36
pop this 6
...
```

BasicTestVME.tst

```
load BasicTest.vm,
output-file BasicTest.out,
compare-to BasicTest.cmp,
// Specifies selected values to output, and their format:
output-list RAM[256]%D1.6.1 RAM[300]%D1.6.1 ...

set sp 256,          // stack pointer
set local 300,       // base address of segment local
set argument 400,    // base address of segment argument
set this 3000,       // base address of segment this
set that 3010,       // base address of segment that

repeat 25 {          // BasicTest.vm requires 25 VM steps
    vmstep;
}

// Outputs the values specified by the output-list
// (stack base + selected values from selected memory segments)
output;
```

The final version of the VM translator will allocate memory segments (local, argument, ...) to the host RAM;

The basic VM translator does not handle memory allocations;

Instead, in project 7 we allocate the memory segments used in the test programs to arbitrary RAM locations, using test script set commands.

Demo



Virtual Machine I: Lecture plan

- Overview
- VM Language: Abstraction
- VM Language: Implementation
- VM Emulator

Standard Mapping

- VM Translator
- Project 7

Standard VM mapping

Background

We've introduced a VM language / model;

The VM can be implemented on various target platforms.

Standard VM mapping

Specifies a standard way for mapping the VM's stack and memory segments on target platforms (for executing VM programs)

Benefits

Compatibility with other tools / libraries that conform to this standard:

- VM emulators, OS
- Testing systems / test scripts
- Etc.

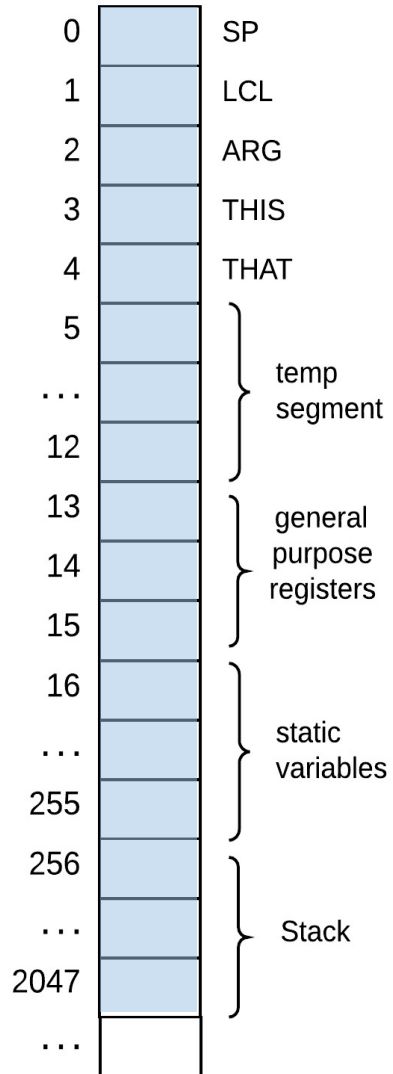
Standard VM mapping on the Hack platform

Hack RAM

0	
1	
2	
3	
4	
5	
...	
12	
13	
14	
15	
16	
...	
255	
256	
...	
2047	
...	

Standard VM mapping on the Hack platform

Hack RAM



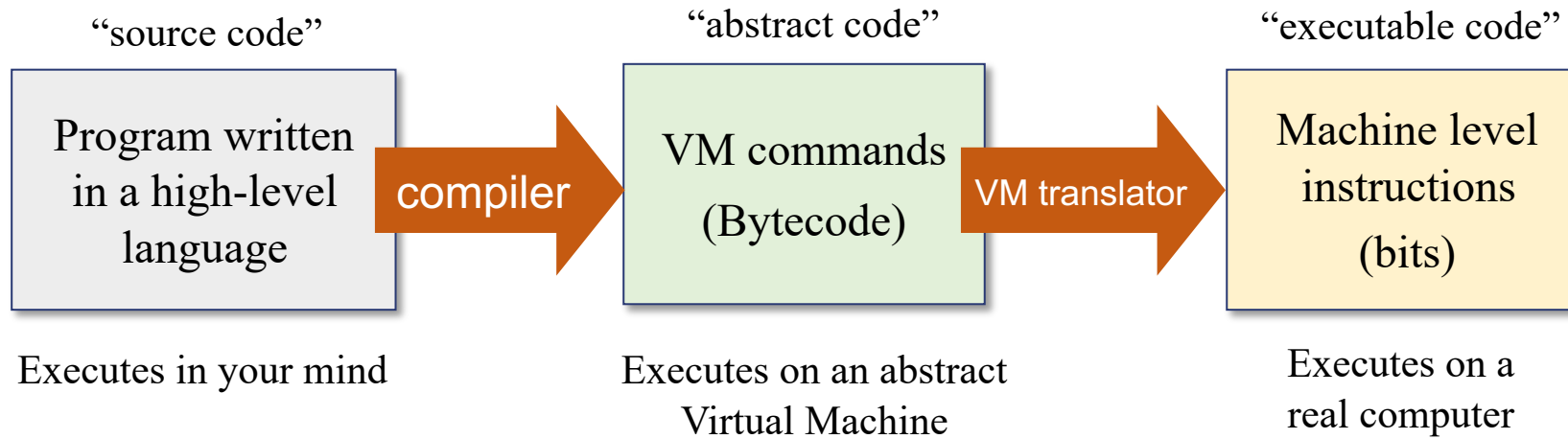
To realize the standard mapping, the assembly code generated by the VM translator must conform to the mapping shown on the left, and use the following symbols:

<i>Symbol</i>	<i>Usage</i>
SP	This predefined symbol points to the memory address within the host RAM just following the address containing the topmost stack value.
LCL, ARG, THIS, THAT	These predefined symbols point, respectively, to the base addresses within the host RAM of the virtual segments <code>local</code> , <code>argument</code> , <code>this</code> , and <code>that</code> of the currently running VM function.
R13–R15	These predefined symbols can be used for any purpose.
<code>Xxx.i</code> symbols	The <code>static</code> segment is implemented as follows: each static variable <code>i</code> in file <code>Xxx.vm</code> is translated into the assembly symbol <code>Xxx.i</code>. In the subsequent assembly process, these symbolic variables will be allocated to the RAM by the Hack assembler.

Virtual Machine I: Lecture plan

- Overview
- VM Language: Abstraction
- VM Language: Implementation
- VM Emulator
- Standard Mapping
- ➔ VM Translator
- Project 7

The Big Picture: Program compilation



The VM translator

VM code (*fileName.vm*)

```
...
push constant 17
push local 2
add
pop argument 1
...
```

VM
translator

Generated assembly code (*fileName.asm*)

```
...
// push constant 17
@17
D=A
... additional assembly code that completes the
implementation of push constant 17
```

The VM translator creates an output `.asm` file, parses the source VM commands line by line, and emits their translation into the output file.

The VM translator

VM code (*fileName.vm*)

```
...  
push constant 17  
push local 2  
add  
pop argument 1  
...
```

VM
translator

Generated assembly code (*fileName.asm*)

```
...  
// push constant 17  
@17  
D=A  
... additional assembly code that completes the  
implementation of push constant 17  
// push local 2  
... generated assembly code that implements push local 2
```

The VM translator creates an output `.asm` file, parses the source VM commands line by line, and emits their translation into the output file.

The VM translator

VM code (*fileName.vm*)

```
...  
push constant 17  
push local 2  
add  
pop argument 1  
...
```

VM
translator

Generated assembly code (*fileName.asm*)

```
...  
// push constant 17  
@17  
D=A  
... additional assembly code that completes the  
implementation of push constant 17  
// push local 2  
... generated assembly code that implements push local 2  
// add  
... generated assembly code that implements add
```

The VM translator creates an output `.asm` file, parses the source VM commands line by line, and emits their translation into the output file.

The VM translator

VM code (*fileName.vm*)

```
...
push constant 17
push local 2
add
pop argument 1
...
```

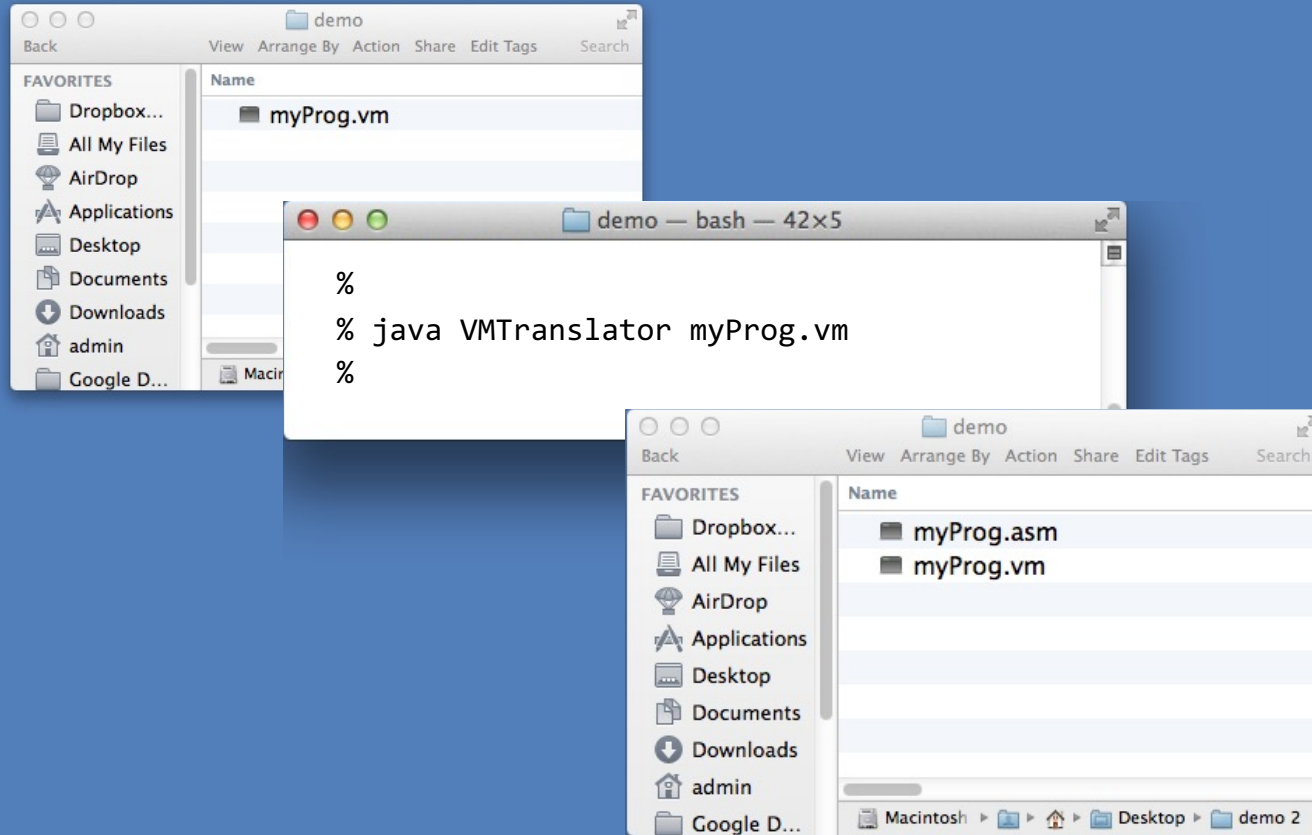
VM
translator

Generated assembly code (*fileName.asm*)

```
...
// push constant 17
@17
D=A
... additional assembly code that completes the
    implementation of push constant 17
// push local 2
... generated assembly code that implements push local 2
// add
... generated assembly code that implements add
// pop argument 1
... generated assembly code that implements push argument 1
...
```

The VM translator creates an output `.asm` file, parses the source VM commands line by line, and emits their translation into the output file.

The VM translator: Usage



The VM translator: Proposed design

Three modules:

Parser: parses each VM command into its lexical elements

CodeWriter: writes the assembly code that implements the parsed command

Main: drives the process (VMTranslator)

Main (VMTranslator)

Input: *fileName.vm*

Output: *fileName.asm*

Main logic

- Constructs a Parser to handle the input file
- Constructs a CodeWriter to handle the output file
- Iterates through the input file, parsing each line and generating code from it, using the services of a Parser and a CodeWriter.

Parser API

- Handles the parsing of a single .vm file
- Reads a VM command, parses the command into its lexical components, and provides convenient access to these components
- Ignores white space and comments

<i>Routine</i>	<i>Arguments</i>	<i>Returns</i>	<i>Function</i>
constructor	input file / stream	—	Opens the input file/stream, and gets ready to parse it.
hasMoreLines	—	boolean	Are there more lines in the input?
advance	—	—	Reads the next command from the input and makes it the <i>current command</i> . This method should be called only if hasMoreLines is true. Initially there is no current command.

(continues in the next slide)

Parser API

- Handles the parsing of a single .vm file
- Reads a VM command, parses the command into its lexical components, and provides convenient access to these components
- Ignores white space and comments

<i>Routine</i>	<i>Arguments</i>	<i>Returns</i>	<i>Function</i>
commandType	—	C_ARITHMETIC, C_PUSH, C_POP, C_LABEL, C_GOTO, C_IF, C_FUNCTION, C_RETURN, C_CALL (constant)	Returns a constant representing the type of the current command. If the current command is an arithmetic-logical command, returns C_ARITHMETIC.
arg1	—	string	Returns the first argument of the current command. In the case of C_ARITHMETIC, the command itself (add, sub, etc.) is returned. Should not be called if the current command is C_RETURN.
arg2	—	int	Returns the second argument of the current command. Should be called only if the current command is C_PUSH, C_POP, C_FUNCTION, or C_CALL.

CodeWriter API

Generates assembly code from the parsed VM command

<i>Routine</i>	<i>Arguments</i>	<i>Returns</i>	<i>Function</i>
constructor	output file / stream	—	Opens an output file / stream and gets ready to write into it.
writeArithmetic	command (string)	—	Writes to the output file the assembly code that implements the given arithmetic-logical command.
WritePushPop	command (C_PUSH or C_POP), segment (string), index (int)	—	Writes to the output file the assembly code that implements the given push or pop command.
close	—	—	Closes the output file.

- This is the essence of the translator: Generating assembly code that realize each VM command
- More routines will be added to this module in Project 8, for handling all the commands of the VM language.

Virtual Machine I: Lecture plan

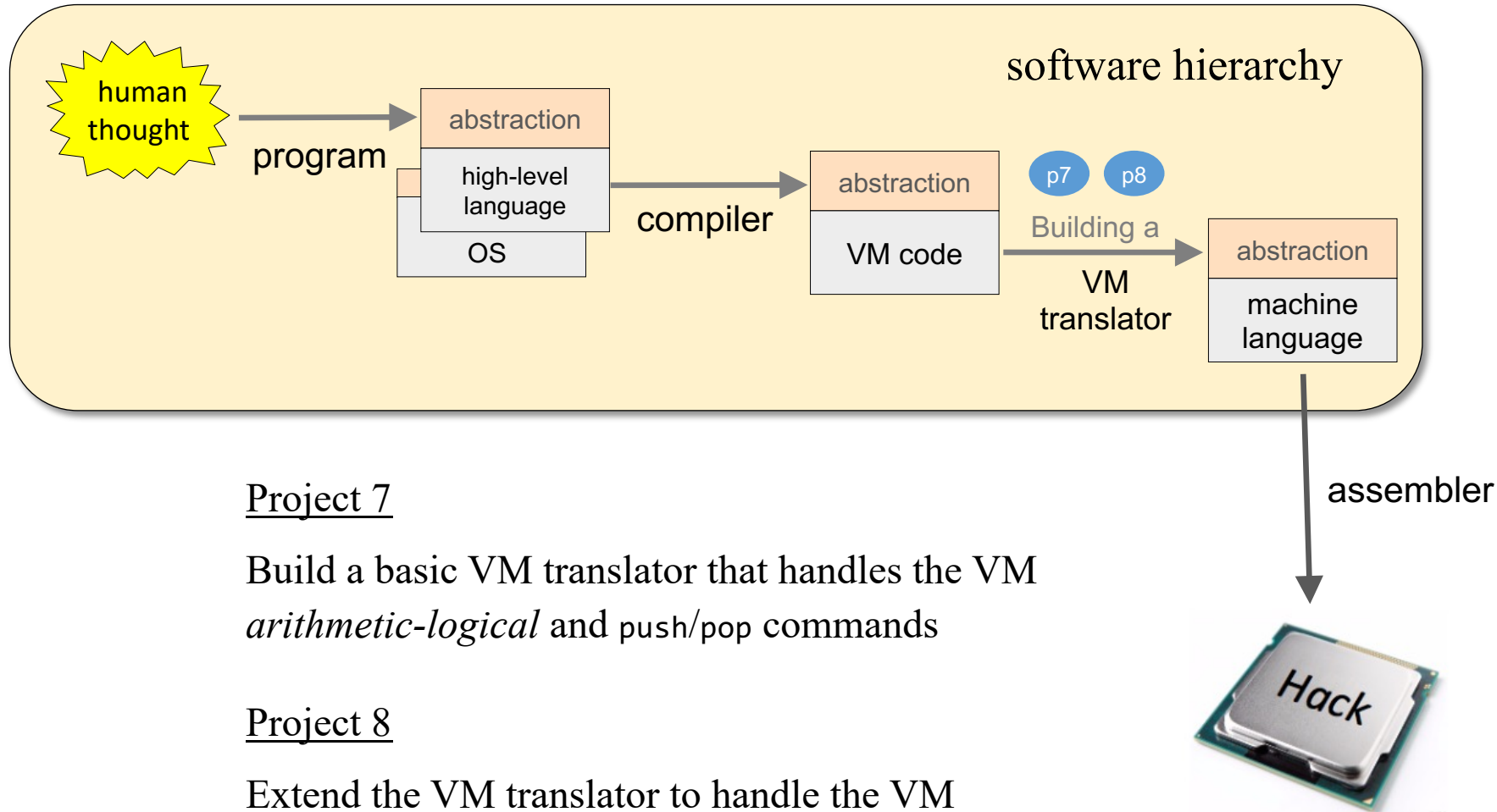
- Overview
- VM Language: Abstraction
- VM Language: Implementation
- VM Emulator
- Standard Mapping
- VM Translator



Project 7

- Perspective

Project 7



Project 7

Build a basic VM translator that handles the VM *arithmetic-logical* and *push/pop* commands

Project 8

Extend the VM translator to handle the VM *branching* and *function* commands.

Project 7

fileName.vm

```
...  
push constant 17  
push local 2  
add  
pop argument 1  
...
```

VM translator

fileName.asm

```
...  
// push constant 17  
@17  
D=A  
...  
// push local 2  
... generated assembly code  
// add  
... generated assembly code  
// pop argument 1  
... generated assembly code  
...
```

Testing option 1: Translate the generated assembly code into machine language:
run the binary code on the Hack computer

➡ Testing option 2 (simpler): Run the generated assembly code on the CPU emulator.

Project 7

Test programs

SimpleAdd.vm

StackTest.vm

BasicTest.vm

PointerTest.vm

StaticTest.vm

Example:

BasicTest.vm

```
...
push constant 510
pop temp 6
push local 0
push that 5
add
push argument 1
sub
...
```

Given

BasicTest.asm

```
...
// push constant 510
@510
D=A
...
```

Generated by your
VM translator

For each test program xxx.vm:

We supply three test files:

xxxVME.tst, xxx.tst and xxx.cmp

0. (optional) Load and run the xxxVME.tst test script in the *VM emulator*; This will cause the emulator to load and execute xxx.vm; Observe the program's operation, and how its stack and segments are managed on the host RAM
1. Use your VM translator to translate xxx.vm; The result will be a file named xxx.asm
2. Inspect the generated code; If there's a problem, fix your translator and go to stage 1
3. Load and run the xxx.tst test script in the *CPU emulator*; This will cause the emulator to load and execute xxx.asm; Inspect the results
4. If there's a problem, fix your translator and go to stage 1.